
Sprawozdanie z Laboratorium: Bazy Danych

Paweł Łoćwin, Paweł Łosowski

Jun 18, 2026

CONTENTS

1	Wprowadzenie	1
2	Spis treści	2
2.1	Rozdział 1: Wstęp oraz linki	2
2.1.1	Wstęp do sprawozdania	2
2.1.2	Wykaz repozytoriów (Linki)	2
2.2	Badania literaturowe	3
2.2.1	Sprzęt dla bazy danych	3
2.2.2	Konfiguracja bazy danych PostgreSQL	3
2.2.3	Kontrola i konserwacja	10
2.2.4	Monitorowanie i diagnostyka	12
2.2.5	Wydajność, skalowanie i replikacja danych	14
2.2.6	Partycjonowanie danych	16
2.2.7	Bezpieczeństwo Baz Danych	16
2.2.8	Kopie zapasowe i odzyskiwanie danych	24
2.3	Rozdział 3: Projektowanie Bazy Danych: System Zarządzania Biblioteką	26
2.3.1	Wybór zagadnienia, opis procesów i danych	26
2.3.2	Prototyp CSV	27
2.3.3	Model Konceptualny (Pojęciowy)	27
2.3.4	Model logiczny i proces normalizacji	28
2.3.5	Model fizyczny bazy danych	29
2.4	Rozdział 4: Implementacja fizyczna i zasilanie bazy danych	30
2.4.1	Definicja fizyczna bazy danych (Zadanie 1)	30
2.4.2	Mechanizm zasilania bazy danych (Zadanie 2)	31
2.4.3	Podsumowanie	32
2.5	Rozdział 5: Komunikacja i operacje na danych	32
2.5.1	Wstęp	32
2.5.2	Pełna specyfikacja stworzonego API	33

WPROWADZENIE

Niniejsza dokumentacja stanowi kompletne sprawozdanie z laboratorium przedmiotu Bazy Danych. Projekt obejmuje całościową analizę procesów związanych z administracją i projektowaniem relacyjnych baz danych.

Dokumentacja została podzielona na pięć głównych sekcji:

- **Rozdział 1:** Wstęp do sprawozdania oraz wykaz repozytoriów z plikami projektowymi i submodułami.
- **Rozdział 2:** Przegląd źródeł akademickich i naukowo-technicznych dotyczących baz danych, ich architektury, optymalizacji i bezpieczeństwa.
- **Rozdział 3:** Praktyczne modelowanie systemu zarządzania bibliotecznym katalogiem wypożyczeń (analiza wymagań, model konceptualny E-R, model logiczny).
- **Rozdział 4:** Realizacja fizycznego schematu bazy danych poprzez skrypty DDL, mechanizmy importu danych oraz automatyzacja zasilania bazy w środowiskach PostgreSQL i SQLite.
- **Rozdział 5:** Zaawansowane funkcje SQL w Pythonie, selekcje, agregacje, złączenia, operatory zbiorowe i podzapytania.

SPIS TREŚCI

2.1 Rozdział 1: Wstęp oraz linki

Autorzy

1. Paweł Łoćwin
2. Paweł Łosowski

2.1.1 Wstęp do sprawozdania

Niniejsze sprawozdanie dokumentuje przebieg prac laboratoryjnych z przedmiotu Bazy Danych. Celem projektu jest praktyczne zastosowanie wiedzy z zakresu projektowania, wdrażania oraz obsługi systemów relacyjnych baz danych.

Praca obejmuje pełen cykl życia bazy: od analizy teoretycznej, poprzez stworzenie modelu pojęciowego i logicznego, aż po fizyczną implementację w środowiskach PostgreSQL i SQLite, a także realizację procesów wsadowego zasilania danymi.

2.1.2 Wykaz repozytoriów (Linki)

Projekt został podzielony na odpowiednie repozytoria w systemie kontroli wersji Git, zapewniając porządek między dokumentacją a fizycznymi plikami środowiska bazodanowego.

1. Główne repozytorium sprawozdania (Dokumentacja Sphinx)

Tutaj znajduje się struktura całej dokumentacji tekstowej, konfiguracja kompilatora Sphinx oraz historia zmian.

- **Link:** <https://github.com/baguetedev/Bazy-Danych-1>

2. Repozytorium z plikami projektowymi

W tym repozytorium umieszczono kody źródłowe, skrypty DDL dla silników PostgreSQL i SQLite, wygenerowany plik bazy oraz pliki CSV wykorzystywane do zasilania bazy danych.

- **Link WWW:** <https://github.com/baguetedev/Bazy-Danych-pliki>
- **Klonowanie SSH:** [git@github.com:baguetedev/Bazy-Danych-pliki.git](https://github.com/baguetedev/Bazy-Danych-pliki.git)

3. Repozytoria reszty grupy

Poniżej znajdują się odnośniki do repozytoriów reszty grupy, które zostały zintegrowane z niniejszym sprawozdaniem w postaci submodułów w Rozdziale 2:

- **Grupa 1 (rozdział_1):** <https://github.com/karaskamil/Sprzet-dla-bazy-danych.git>
- **Grupa 2 (rozdział_2):** https://github.com/Youarecheck/Bazy_Danych_Tematyczne_Repo_MK.git
- **Grupa 3 (rozdział_3):** <https://github.com/pawlos1337/Bazy-danych-temat.git>
- **Grupa 4 (rozdział_4):** https://github.com/OskarProgrammer/monitorowanie_i_diagnostyka.git
- **Grupa 5 (rozdział_5):** <https://github.com/KMachoK/Tematyczne.git>
- **Grupa 6 (rozdział_6):** <https://github.com/domino0472/Partycjonowani-Danych>
- **Grupa 7 (rozdział_7):** <https://github.com/oski486/BazyDanych-Subject.git>
- **Grupa 8 (rozdział_8):** <https://github.com/Koko9077/Kopie-zapasowe-i-odzyskiwanie-danych.git>

2.2 Badania literaturowe

2.2.1 Sprzęt dla bazy danych

2.2.2 Konfiguracja bazy danych PostgreSQL

Wstęp

Rozdział opisuje konfigurację połączenia z bazą danych **PostgreSQL** – dojrzałym, open-source’owym systemem zarządzania relacyjnymi bazami danych (RDBMS), który wyróżnia się pełną zgodnością ze standardem SQL oraz silnym naciskiem na rozszerzalność i niezawodność.

Wymagania:

- dostęp do instancji PostgreSQL,
- zmienne środowiskowe,
- narzędzia projektowe (np. `psql`, `pg_isready`).

Podstawowe parametry

Każde połączenie z PostgreSQL wymaga:

- hosta i portu (domyślnie `localhost` i `5432`),
- nazwy bazy danych,
- użytkownika i hasła.

Dane wrażliwe (np. hasła) przechowuj w **zmiennych środowiskowych** – to oddziela konfigurację od kodu. Pamiętaj: zmienne środowiskowe nie są mechanizmem bezpieczeństwa; w produkcji uzupełnij je o dedykowane zarządzanie sekretami (np. HashiCorp Vault, AWS Secrets Manager).

Lokalizacja i struktura katalogów PostgreSQL

Pliki konfiguracyjne PostgreSQL znajdują się w katalogu danych (PGDATA).

Główne pliki konfiguracyjne:

- `postgresql.conf` – podstawowa konfiguracja serwera (port, pamięć, logi)
- `pg_hba.conf` – kontrola dostępu (kto i skąd może się łączyć)
- `pg_ident.conf` – mapowanie użytkowników systemowych na role PostgreSQL

Domyślne lokalizacje:

Table 1: Domyślne lokalizacje

System	Katalog danych
Linux (RPM/DEB)	<code>/var/lib/pgsql/data/</code> lub <code>/var/lib/postgresql/*/main/</code>
Linux (kompilacja źródłowa)	<code>/usr/local/pgsql/data/</code>
Windows	<code>C:\Program Files\PostgreSQL\<version>\data\</code>
Docker	<code>/var/lib/postgresql/data/</code> (w kontenerze)

Zmiana katalogu danych:

```
# Inicjalizacja nowego katalogu
initdb -D /ścieżka/do/katalogu

# Uruchomienie z niestandardowym PGDATA
pg_ctl -D /ścieżka/do/katalogu start
```

Struktura katalogu danych:

```
$PGDATA/
├── postgresql.conf  # główny plik konfiguracyjny
├── pg_hba.conf      # polityka dostępu
└── pg_ident.conf    # mapowanie tożsamości
```

(continues on next page)

(continued from previous page)

```
└─ base/          # rzeczywiste bazy danych
└─ global/        # tabele systemowe
└─ log/           # logi serwera
└─ pg_wal/        # Write-Ahead Log (WAL)
```

Sprawdzenie aktualnej lokalizacji:

```
SHOW data_directory;
SHOW config_file;
SHOW hba_file;
```

Środowiska i profile

Przełączanie środowisk PostgreSQL (dev/test/prod) realizuj przez:

- zmienne środowiskowe (np. PGHOST, PGPORT, PGDATABASE, PGUSER, PGPASSWORD),
- profile konfiguracyjne,
- osobne pliki .env dla każdego środowiska.

Automatyzacja

Zalecenia dla PostgreSQL:

- walidacja konfiguracji przy starcie za pomocą pg_isready,
- skrypty sprawdzające kompletność zmiennych środowiskowych,
- integracja z CI/CD (np. GitHub Actions, GitLab CI).

Dokumentację generuj automatycznie (Sphinx).

Przykłady

PostgreSQL – zmienne środowiskowe (.env)

```
PGHOST=localhost
PGPORT=5432
PGDATABASE=aplikacja
PGUSER=app_user
PGPASSWORD=${DB_PASSWORD}
```

PostgreSQL – URL połączenia

```
DATABASE_URL=postgresql://${PGUSER}:${PGPASSWORD}@${PGHOST}:${PGPORT}/${PGDATABASE}
```

PostgreSQL – sprawdzenie połączenia

```
pg_isready -h ${PGHOST} -p ${PGPORT} -U ${PGUSER} -d ${PGDATABASE}
```

Najlepsze praktyki

1. Nie przechowuj sekretów w repozytorium.
2. Stosuj .env.example zamiast rzeczywistych danych.
3. Używaj standardowych zmiennych środowiskowych PostgreSQL (PG*).
4. Waliduj konfigurację przed uruchomieniem (pg_isready).
5. W środowiskach produkcyjnych używaj połączeń SSL/TLS (sslmode=require).

Podsumowanie

Poprawna konfiguracja połączenia z PostgreSQL zwiększa bezpieczeństwo i przenośność aplikacji między środowiskami. Standardowe zmienne środowiskowe oraz narzędzia takie jak `pg_isready` ułatwiają automatyzację i walidację.

Szczegółowe omówienie plików konfiguracyjnych

Po omówieniu podstawowych wymagań środowiskowych można przejść do szczegółowej konfiguracji PostgreSQL. W kolejnych sekcjach przedstawiono najważniejsze parametry konfiguracyjne dostępne w plikach:

- `postgresql.conf` – konfiguracja działania serwera,
- `pg_hba.conf` – kontrola dostępu do bazy danych,
- `pg_ident.conf` – mapowanie użytkowników systemowych na role PostgreSQL.

postgresql.conf

Plik `postgresql.conf` jest głównym plikiem konfiguracyjnym klastra PostgreSQL. Zawiera ustawienia wpływające na działanie serwera, wydajność, bezpieczeństwo, logowanie oraz replikację.

Lokalizację aktywnego pliku można sprawdzić poleceniem:

```
SHOW config_file;
```

Najważniejsze parametry

Połączenia sieciowe

listen_addresses

Określa adresy IP, na których PostgreSQL nasłuchuje połączeń.

Najczęstsze wartości:

- `localhost` – tylko połączenia lokalne,
- `*` – wszystkie interfejsy sieciowe,
- lista adresów, np. `'192.168.1.10,localhost'`.

port

Port TCP używany przez PostgreSQL.

Domyślna wartość:

```
port = 5432
```

max_connections

Maksymalna liczba jednoczesnych połączeń z bazą danych.

Zbyt wysoka wartość może zwiększać zużycie pamięci.

Pamięć i wydajność

shared_buffers

Ilość pamięci RAM używanej przez PostgreSQL do buforowania danych.

Typowe wartości:

- 128MB – środowiska testowe,
- 25% pamięci RAM – środowiska produkcyjne.

work_mem

Ilość pamięci wykorzystywanej przez operacje sortowania i zapytania.

Parametr jest przydzielany dla pojedynczej operacji.

maintenance_work_mem

Pamięć używana podczas operacji administracyjnych, np.:

- `VACUUM`,
- `CREATE INDEX`,
- `ALTER TABLE`.

effective_cache_size

Szacowany rozmiar pamięci podręcznej dostępnej dla PostgreSQL.

Parametr pomaga optymalizatorowi zapytań wybierać odpowiednie plany wykonania.

Logowanie**logging_collector**

Włącza zapisywanie logów do plików.

Dostępne wartości:

- on,
- off.

log_directory

Katalog przechowywania logów.

log_filename

Wzorzec nazw plików logów.

Przykład:

```
log_filename = 'postgresql-%Y-%m-%d.log'
```

log_min_duration_statement

Określa minimalny czas wykonania zapytania (w ms), po którym zostanie ono zapisane w logach.

Przykład:

```
log_min_duration_statement = 1000
```

Wartość 1000 oznacza logowanie zapytań trwających dłużej niż 1 sekundę.

Bezpieczeństwo**password_encryption**

Algorytm szyfrowania haseł użytkowników.

Zalecana wartość:

```
password_encryption = scram-sha-256
```

ssl

Włącza obsługę połączeń SSL/TLS.

Dostępne wartości:

- on,
- off.

ssl_cert_file / ssl_key_file

Ścieżki do certyfikatu i klucza prywatnego SSL.

Replikacja i WAL**wal_level**

Określa poziom szczegółowości Write-Ahead Log (WAL).

Dostępne wartości:

- minimal,
- replica,
- logical.

max_wal_size

Maksymalny rozmiar plików WAL przed wykonaniem checkpointu.

archive_mode

Włącza archiwizację WAL.

archive_command

Polecenie wykonywane podczas archiwizacji plików WAL.

Przykładowa konfiguracja

```
listen_addresses = '*'
port = 5432
max_connections = 200

shared_buffers = 1GB
work_mem = 16MB
maintenance_work_mem = 256MB

logging_collector = on
log_directory = 'log'

ssl = on
password_encryption = scram-sha-256

wal_level = replica
max_wal_size = 2GB
```

Weryfikacja konfiguracji

Po zmianie parametrów konfigurację można przeładować bez restartu serwera:

```
SELECT pg_reload_conf();
```

Niektóre parametry wymagają pełnego restartu PostgreSQL.
Aktualne wartości parametrów można sprawdzić poleceniem:

```
SHOW shared_buffers;
SHOW wal_level;
SHOW max_connections;
```

pg_hba.conf

Plik `pg_hba.conf` (Host-Based Authentication) odpowiada za kontrolę dostępu do serwera PostgreSQL.
Definiuje:

- kto może połączyć się z bazą danych,
- z jakiego adresu IP,
- do jakiej bazy danych,
- przy użyciu jakiej metody uwierzytelniania.

Lokalizację aktywnego pliku można sprawdzić poleceniem:

```
SHOW hba_file;
```

Składnia wpisu

Każdy wpis w `pg_hba.conf` ma następującą strukturę:

TYPE	DATABASE	USER	ADDRESS	METHOD
------	----------	------	---------	--------

Znaczenie pól:

TYPE

Typ połączenia.

Dostępne wartości:

- `local` – połączenia lokalne (Unix socket),

- `host` – połączenia TCP/IP,
- `hostssl` – połączenia TCP/IP z SSL,
- `hostnoss1` – połączenia TCP/IP bez SSL.

DATABASE

Nazwa bazy danych, do której użytkownik może się połączyć.

Możliwe wartości:

- konkretna nazwa bazy,
- `all` – wszystkie bazy,
- `sameuser` – baza o nazwie zgodnej z użytkownikiem.

USER

Użytkownik lub rola PostgreSQL.

ADDRESS

Adres IP lub zakres sieci w notacji CIDR.

Przykłady:

- `127.0.0.1/32`,
- `192.168.1.0/24`,
- `0.0.0.0/0` – wszystkie adresy.

METHOD

Metoda uwierzytelniania użytkownika.

Najczęściej używane metody uwierzytelniania**trust**

Zezwala na połączenie bez uwierzytelniania.

Zalecane wyłącznie w środowiskach testowych.

reject

Odrzuca połączenie.

md5

Uwierzytelnianie przy użyciu hasła MD5.

Metoda starsza i mniej bezpieczna niż SCRAM.

scram-sha-256

Nowoczesna metoda uwierzytelniania oparta na SCRAM.

Zalecana dla nowych instalacji PostgreSQL.

peer

Uwierzytelnianie na podstawie użytkownika systemowego.

Najczęściej używane dla lokalnych połączeń Linux/Unix.

ident

Mapowanie użytkownika systemowego przy użyciu `pg_ident.conf`.

Przykładowa konfiguracja

```
# Połączenia lokalne
local  all                                postgres                                peer

# Połączenia localhost IPv4
host   all                                all                                127.0.0.1/32                        scram-sha-256

# Połączenia localhost IPv6
host   all                                all                                ::1/128                             scram-sha-256

# Sieć lokalna
host   aplikacja                          app_user                          192.168.1.0/24                      scram-sha-256
```

Najlepsze praktyki

1. Używaj `scram-sha-256` zamiast `md5`.
2. Ograniczaj dostęp do konkretnych adresów IP.
3. Nie używaj `trust` w środowiskach produkcyjnych.
4. Stosuj `hostssl` dla połączeń zdalnych.
5. Po zmianie konfiguracji wykonuj reload konfiguracji:

```
SELECT pg_reload_conf();
```

pg_ident.conf

Plik `pg_ident.conf` umożliwia mapowanie użytkowników systemowych na role PostgreSQL.

Najczęściej używany jest razem z metodami uwierzytelniania:

- `peer`,
- `ident`.

Lokalizację aktywnego pliku można sprawdzić poleceniem:

```
SHOW ident_file;
```

Zastosowanie

Mechanizm mapowania pozwala określić, który użytkownik systemowy może zostać powiązany z konkretną rolą PostgreSQL.

Jest to szczególnie przydatne:

- w środowiskach Linux/Unix,
- przy automatyzacji administracyjnej,
- dla lokalnych połączeń bez użycia haseł.

Składnia wpisu

MAPNAME	SYSTEM-USERNAME	PG-USERNAME
---------	-----------------	-------------

Znaczenie pól:

MAPNAME

Nazwa mapowania wykorzystywana w `pg_hba.conf`.

SYSTEM-USERNAME

Użytkownik systemu operacyjnego.

PG-USERNAME

Rola PostgreSQL, na którą użytkownik ma zostać zmapowany.

Przykładowa konfiguracja

# MAPNAME	SYSTEM-USERNAME	PG-USERNAME
admin_map	postgres	postgres
admin_map	deploy	db_admin
app_map	appuser	aplikacja

Przykład użycia w `pg_hba.conf`:

```
local all all peer map=admin_map
```

W powyższym przykładzie użytkownicy systemowi zdefiniowani w `admin_map` mogą logować się jako odpowiadające im role PostgreSQL.

Najlepsze praktyki

1. Ograniczaj mapowania wyłącznie do wymaganych użytkowników.
2. Nie mapuj użytkowników systemowych na role superuser bez uzasadnienia.
3. Stosuj osobne role administracyjne i aplikacyjne.
4. Dokumentuj wszystkie mapowania użytkowników.
5. Regularnie przeglądaj konfigurację dostępu.

Opracowanie

Autor: Michał Kraus **Przedmiot:** Bazy danych **Data:** maj 2026

2.2.3 Kontrola i konserwacja

Autorzy

1. Paweł Łoćwin
2. Paweł Łosowski

Wprowadzenie i planowanie konserwacji

Współczesne systemy relacyjnych baz danych to wysoce skomplikowane środowiska, które wymagają ciągłego i proaktywnego nadzoru. Aby zagwarantować wysoką dostępność i niezawodność, niezbędne jest rygorystyczne planowanie procesów konserwacyjnych, które opiera się na odpowiedzi na trzy kluczowe pytania:

- **Kiedy?** Operacje inwazyjne (np. pełna przebudowa tabel i indeksów) powinny być planowane w tzw. oknach serwisowych, czyli w godzinach najniższego obciążenia systemu (zwykle w nocy lub w weekendy). Lżejsze prace konserwacyjne mogą odbywać się w tle.
- **W jakim zakresie?** Należy precyzyjnie określić, czy konserwacji wymaga cała instancja, pojedyncza baza danych, czy tylko wytypowane, najszybciej rosnące tabele. Skupienie się na najbardziej obciążonych obszarach oszczędza zasoby sprzętowe.
- **Jak?** Konserwacja powinna być maksymalnie zautomatyzowana (np. z wykorzystaniem systemowego demona CRON lub rozszerzenia *pg_cron*). Ręczne interwencje administratora powinny być zarezerwowane wyłącznie dla sytuacji awaryjnych i niestandardowych problemów wydajnościowych.

Zarządzanie stanem serwera i sesjami

Podstawą kontroli nad środowiskiem bazodanowym jest zarządzanie cyklem życia samej usługi. Uruchamianie, zatrzymywanie i restartowanie serwera bazy danych realizuje się najczęściej za pośrednictwem menedżera usług systemu operacyjnego (np. polecenia *systemctl start/stop/restart postgresql*) lub za pomocą dedykowanego narzędzia wiersza poleceń *pg_ctl*.

Podczas wdrażania krytycznych aktualizacji lub w sytuacjach awaryjnych, konieczne jest sprawne zarządzanie ruchem użytkowników:

- **Zapobieganie nowym połączeniom:** Administrator może tymczasowo zablokować możliwość logowania się do konkretnej bazy danych poprzez zmianę jej metadanych. Pozwala to na “odcięcie” aplikacji bez wyłączania całego serwera.
- **Ograniczanie i rozłączanie użytkowników:** Istniejące, aktywne sesje, które np. zawiesiły się lub blokują ważne operacje, można siłowo przerwać, zwalniając tym samym zablokowane zasoby.

```
-- Zapobieganie nawiązywaniu nowych połączeń do bazy danych
```

```
ALTER DATABASE biblioteka_db ALLOW_CONNECTIONS = false;
```

```
-- Siłowe rozłączenie wszystkich aktywnych użytkowników (poza aktualną sesją)
```

```
SELECT pg_terminate_backend(pid)
```

```
FROM pg_stat_activity
```

```
WHERE datname = 'biblioteka_db' AND pid <> pg_backend_pid();
```

Proces Vacuum

Architektura PostgreSQL opiera się na mechanizmie MVCC (Multi-Version Concurrency Control). Oznacza to, że instrukcje aktualizacji (UPDATE) i usuwania (DELETE) nie niszczą fizycznie starych danych, lecz tworzą tzw. “martwe krotki” (dead tuples). Do zarządzania nimi służy proces czyszczenia:

- **VACUUM:** Standardowa operacja zwalniająca miejsce po martwych wierszach, czyniąca je dostępnymi dla nowych danych. Nie zmniejsza ona fizycznego rozmiaru pliku na dysku i nie blokuje możliwości jednoczesnego odczytu i zapisu tabeli.
- **VACUUM FULL:** Inwazyjna i ciężka operacja, która fizycznie przebudowuje tabelę, faktycznie zmniejszając jej rozmiar i zwracając wolne miejsce do systemu operacyjnego. Wymaga ekskluzywnej blokady (lock), całkowicie odcinając aplikację od modyfikowanej tabeli na czas trwania procesu.
- **AUTOVACUUM:** Zautomatyzowany proces działający w tle, który regularnie monitoruje poziom zmian w tabelach i samoczynnie wyzwała standardowy Vacuum, zapobiegając niekontrolowanemu puchnięciu bazy danych (table bloat).

Zarządzanie transakcjami i schematy (SCHEMA)

Kolejnym aspektem administracji jest dbanie o logiczną strukturę bazy oraz spójność operacji.

SCHEMA (Schematy) to wirtualne przestrzenie nazw (przypominające katalogi w systemie operacyjnym), służące do logicznego grupowania tabel, widoków i funkcji. Ich główne zastosowania to: * Izolacja danych różnych mikrousług lub aplikacji wewnątrz jednej wspólnej bazy danych. * Implementacja architektury wielodostępowej (multi-tenant), gdzie każdy klient aplikacji obsługiwany jest w dedykowanym, oddzielnym schemacie. * Ułatwienie i zbiorcze zarządzanie uprawnieniami dostępu.

Zarządzanie transakcjami to proces kontrolowania instrukcji (BEGIN, COMMIT, ROLLBACK) w taki sposób, aby zachować zgodność z regułami ACID. Administrator musi zwracać szczególną uwagę na zapobieganie tzw. długim transakcjom (long-running transactions). Otwarta i porzucona transakcja powstrzymuje proces Vacuum przed usuwaniem martwych krotek, co prowadzi do drastycznego spadku wydajności całej bazy.

```
-- Przykład logiki transakcyjnej z wykorzystaniem konkretnego schematu
BEGIN;
INSERT INTO finanse.historia (kwota, opis) VALUES (200, 'Opłata serwisowa');
UPDATE finanse.konta SET saldo = saldo - 200 WHERE id_klienta = 5;
COMMIT;
```

Zarządzanie indeksami

Indeksy są fundamentalne dla wydajnego wyszukiwania danych, jednak ich utrzymanie kosztuje — spowalniają one modyfikacje danych (DML) i konsumują miejsce na dysku.

Prawidłowe zarządzanie indeksami polega na stałym monitorowaniu ich wykorzystania. Należy identyfikować i usuwać indeksy, które nie są wykorzystywane przez planistę zapytań (tzw. unused indexes). Dodatkowo, podobnie jak tabele, indeksy ulegają fragmentacji. Z tego powodu administrator powinien regularnie planować operację **REINDEX**, która od nowa buduje strukturę drzewa indeksu, przywracając mu optymalny rozmiar i maksymalną wydajność. W systemach o wysokiej dostępności stosuje się przebudowę w trybie **CONCURRENTLY**, która nie przerywa pracy aplikacji.

Podsumowanie

Prawidłowa kontrola i konserwacja środowiska bazodanowego to wielowymiarowy proces, wymagający głębokiego zrozumienia zarówno architektury logicznej, jak i fizycznej serwera.

Stabilność i wydajność systemu zależy w równej mierze od prawidłowego zarządzania stanem usługi i połączeniami, dogłębnego planowania okien serwisowych, jak i proaktywnego zarządzania mechanizmem Vacuum i transakcjami. Wykorzystanie schematów do strukturyzacji danych oraz regularne dbanie o kondycję indeksów pozwala na zbudowanie środowiska, które będzie skalowalne, przewidywalne i odporne na awarie w trudnych, produkcyjnych warunkach.

Autorzy

1. Paweł Łoćwin

2. Paweł Łosowski

2.2.4 Monitorowanie i diagnostyka

Autorzy

1. Oskar Wrona
2. Kamil Lewandowski
3. Adam Tarkowski

Wstęp

Monitorowanie i diagnostyka baz danych obejmują zestaw działań, które pozwalają administratorowi lub zespołowi utrzymania sprawdzić, czy system działa poprawnie, bezpiecznie i wydajnie. W praktyce nie chodzi jedynie o obserwowanie pojedynczych parametrów serwera, ale o łączenie informacji pochodzących z wielu warstw: samej bazy danych, systemu operacyjnego, aplikacji oraz narzędzi zewnętrznych. Dzięki temu można szybciej wykrywać przeciążenia, błędy konfiguracji, problemy z zapytaniami SQL, nieprawidłowe uprawnienia oraz awarie sprzętowe lub sieciowe.

W systemach bazodanowych szczególnie ważne jest to, że wiele problemów nie jest od razu widocznych dla użytkownika końcowego. Przykładowo zapytania mogą stopniowo wykonywać się coraz wolniej, liczba aktywnych sesji może rosnąć, a logi mogą zawierać powtarzające się ostrzeżenia. Bez monitorowania takie sygnały są łatwe do przeoczenia. Diagnostyka pozwala natomiast przejść od samego wykrycia problemu do ustalenia jego przyczyny, np. przez analizę sesji, blokad, planów zapytań, dzienników błędów lub zużycia zasobów systemowych.

Sesje i użytkownicy

Jednym z podstawowych obszarów monitorowania bazy danych jest obserwowanie aktywnych sesji i użytkowników. Sesja oznacza połączenie użytkownika lub aplikacji z serwerem bazy danych. W ramach sesji wykonywane są zapytania, transakcje oraz operacje administracyjne. Analiza sesji pozwala sprawdzić między innymi, kto jest aktualnie połączony z bazą, z jakiego hosta pochodzi połączenie, jakie zapytanie jest wykonywane oraz czy sesja jest aktywna, bezczynna albo zablokowana.

W wielu systemach zarządzania bazami danych dostępne są specjalne widoki lub narzędzia administracyjne służące do obserwowania bieżącej aktywności. Na przykład PostgreSQL udostępnia mechanizmy monitorowania aktywności bazy danych oraz widoki statystyczne, takie jak `pg_stat_activity`, które pozwalają sprawdzać informacje o procesach serwera i wykonywanych zapytaniach¹. W MySQL podobną rolę może pełnić Performance Schema, które gromadzi dane diagnostyczne o pracy serwera i może być wykorzystywane do analizy wydajności². Monitorowanie sesji jest istotne zarówno z punktu widzenia wydajności, jak i bezpieczeństwa. Zbyt duża liczba jednoczesnych połączeń może prowadzić do przeciążenia serwera, a długo działające transakcje mogą blokować inne operacje. Z kolei nietypowe połączenia, np. z nieznanych adresów lub wykonywane poza normalnymi godzinami pracy, mogą wskazywać na błąd konfiguracji albo próbę nieautoryzowanego dostępu.

Śledzenie dostępu użytkowników do tabel

Kolejnym ważnym elementem jest kontrola tego, którzy użytkownicy uzyskują dostęp do określonych tabel i jakie operacje wykonują. W bazach danych przechowywane są często dane wrażliwe, dlatego sama informacja o tym, że użytkownik posiada konto w systemie, nie jest wystarczająca. Należy również wiedzieć, czy jego działania są zgodne z nadanymi uprawnieniami oraz z zasadami bezpieczeństwa obowiązującymi w organizacji.

Śledzenie dostępu może obejmować operacje odczytu danych, modyfikacji rekordów, usuwania danych, tworzenia nowych obiektów oraz zmian w uprawnieniach. W praktyce realizuje się to za pomocą mechanizmów audytu, dzienników zapytań, wyzwalaczy, narzędzi klasy Database Activity Monitoring lub funkcji wbudowanych w

¹ PostgreSQL Documentation, *Monitoring Database Activity* oraz *The Cumulative Statistics System*, <https://www.postgresql.org/docs/current/monitoring.html>

² MySQL Documentation, *Using the Performance Schema to Diagnose Problems*, <https://dev.mysql.com/doc/refman/8.2/en/performance-schema-examples.html>

konkretny system bazodanowy. Przykładowo Oracle Database udostępnia mechanizmy audytu, a nowsze wersje zalecają korzystanie z unified auditing zamiast starszego audytu tradycyjnego³.

Takie śledzenie jest potrzebne nie tylko przy wykrywaniu incydentów bezpieczeństwa. Pomaga również w spełnianiu wymagań formalnych, analizie błędów użytkowników oraz odtwarzaniu przebiegu zdarzeń po awarii. Przykładowo, jeśli w tabeli pojawiły się nieprawidłowe dane, historia dostępu może pomóc ustalić, kiedy doszło do zmiany i która aplikacja lub który użytkownik ją wykonał.

Błędy dziennika, logi i raporty

Dzienniki zdarzeń, czyli logi, są jednym z najważniejszych źródeł informacji diagnostycznych. Serwer bazy danych może zapisywać w nich komunikaty o błędach, ostrzeżeniach, nieudanych logowaniach, problemach z zapytaniami, restartach, zmianach konfiguracji lub przekroczeniu określonych progów wydajnościowych. Analiza logów pozwala wykrywać problemy, które nie zawsze są widoczne w aplikacji klienckiej.

W zależności od systemu bazodanowego logi mogą mieć różną postać. PostgreSQL obsługuje różne metody logowania komunikatów serwera, między innymi `stderr`, `csvlog`, `jsonlog` oraz `syslog`; w systemie Windows możliwe jest także użycie dziennika zdarzeń⁴. MySQL udostępnia między innymi `general query log`, który może rejestrować połączenia i zapytania otrzymywane przez serwer, oraz `slow query log`, używany do identyfikowania zapytań wykonujących się zbyt długo⁵.

Raporty diagnostyczne są rozwinięciem surowych logów. Zamiast ręcznie analizować tysiące wpisów, administrator może korzystać z raportów podsumowujących liczbę błędów, najwolniejsze zapytania, częstotliwość nieudanych logowań lub zmiany obciążenia w czasie. Raporty są szczególnie przydatne w pracy zespołowej, ponieważ pozwalają dokumentować problemy i porównywać stan systemu przed oraz po wprowadzeniu zmian optymalizacyjnych.

Monitorowanie na poziomie systemu operacyjnego

Baza danych działa na konkretnym systemie operacyjnym i korzysta z jego zasobów. Dlatego diagnostyka nie może ograniczać się wyłącznie do samego silnika bazy danych. Wydajność zapytań może zależeć od procesora, pamięci RAM, operacji wejścia/wyjścia na dysku, sieci, liczby procesów oraz ogólnego obciążenia systemu.

W systemach Linux często wykorzystuje się narzędzia takie jak `iostat`, `htop` oraz `vmstat`. Narzędzie `iostat` pozwala obserwować wykorzystanie procesora i urządzeń wejścia/wyjścia, co jest szczególnie ważne przy bazach danych intensywnie korzystających z dysku. `htop` umożliwia wygodne śledzenie procesów i zużycia zasobów w czasie rzeczywistym. `vmstat` pokazuje między innymi informacje o pamięci, procesach, wymianie danych z dyskiem oraz obciążeniu procesora.

W systemach Microsoft Windows podobną rolę pełnią Menedżer zadań oraz Monitor wydajności. Pozwalają one obserwować zużycie procesora, pamięci, dysków, sieci oraz usług działających w tle. W środowiskach produkcyjnych takie dane są często zbierane automatycznie i zestawiane z metrykami pochodzącymi z bazy danych. Dzięki temu można odróżnić problem wynikający z nieoptymalnego zapytania SQL od problemu spowodowanego np. przeciążeniem dysku lub brakiem pamięci.

Monitorowanie serwera

Ostatnim poziomem jest monitorowanie całego serwera lub infrastruktury, w której działa baza danych. Obejmuje ono dostępność usług, czas odpowiedzi, zużycie zasobów, stan dysków, wykorzystanie sieci, działanie kopii zapasowych oraz wysyłanie powiadomień w przypadku awarii. W tym celu stosuje się narzędzia takie jak Nagios, Grafana, Prometheus, Zabbix lub inne systemy monitoringu.

Nagios jest przykładem narzędzia używanego do monitorowania usług i hostów, sprawdzania ich stanu oraz informowania administratorów o problemach⁶. Grafana z kolei służy przede wszystkim do wizualizacji danych monitorujących w formie dashboardów. Panele Grafany mogą prezentować metryki w postaci wykresów, tabel i innych

³ Oracle Documentation, *AUDIT_TRAIL* oraz informacje o unified auditing, https://docs.oracle.com/en/database/oracle/oracle-database/21/refrn/AUDIT_TRAIL.html

⁴ PostgreSQL Documentation, *Error Reporting and Logging*, <https://www.postgresql.org/docs/current/runtime-config-logging.html>

⁵ MySQL Documentation, *The General Query Log* oraz *The Slow Query Log*, <https://dev.mysql.com/doc/en/query-log.html>

⁶ Nagios Open Source Documentation, <https://www.nagios.org/documentation/>

wizualizacji, a moduł alertów pozwala reagować na określone zdarzenia lub przekroczenie progów⁷.

Monitorowanie serwera jest szczególnie ważne w środowiskach, w których baza danych stanowi element większego systemu. Awaria bazy może wynikać nie tylko z błędu samego silnika, ale również z problemu sieciowego, zapełnionego dysku, braku pamięci, błędnej konfiguracji systemu lub nieudanego wdrożenia aplikacji. Centralne monitorowanie pozwala zebrać te informacje w jednym miejscu i szybciej wskazać źródło problemu.

Znaczenie monitorowania i diagnostyki w bazach danych

Monitorowanie i diagnostyka są ważne, ponieważ wpływają na trzy kluczowe obszary: dostępność, wydajność i bezpieczeństwo danych. Dostępność oznacza, że baza danych jest osiągalna dla użytkowników i aplikacji. Wydajność oznacza, że zapytania wykonują się w akceptowalnym czasie. Bezpieczeństwo oznacza natomiast, że dostęp do danych jest kontrolowany, a podejrzane działania mogą zostać wykryte i przeanalizowane.

W dobrze utrzymanym środowisku bazodanowym monitorowanie powinno działać stale, a nie dopiero wtedy, gdy pojawi się awaria. Regularne obserwowanie sesji, logów, dostępu do tabel, parametrów systemu operacyjnego i stanu serwera pozwala wykrywać problemy na wczesnym etapie. Diagnostyka pozwala natomiast ustalić przyczyny tych problemów i zaplanować odpowiednie działania, takie jak optymalizacja zapytań, zmiana konfiguracji, zwiększenie zasobów, poprawa uprawnień lub wdrożenie dodatkowych zabezpieczeń.

Podsumowanie

Monitorowanie i diagnostyka baz danych tworzą wielowarstwowy proces kontroli działania systemu. Na poziomie bazy danych analizuje się sesje, użytkowników, zapytania, blokady, dostęp do tabel oraz logi. Na poziomie systemu operacyjnego obserwuje się zużycie procesora, pamięci, dysku i sieci. Na poziomie serwera wykorzystuje się narzędzia monitorujące, które zbierają metryki, tworzą wykresy oraz wysyłają alerty. Dopiero połączenie tych perspektyw daje pełny obraz stanu systemu i pozwala skutecznie reagować na błędy, spadki wydajności oraz potencjalne incydenty bezpieczeństwa.

Źródła

2.2.5 Wydajność, skalowanie i replikacja danych

Autorzy:

1. Olaf Chomicki
2. Konrad Machowski
3. Wiktor Wydrzyński

Wstęp

W erze gwałtownego wzrostu ilości generowanych informacji, stabilne działanie systemów bazodanowych stało się fundamentem nowoczesnych aplikacji. Wydajność, skalowanie oraz replikacja to trzy ściśle powiązane ze sobą filary, które decydują o tym, czy baza danych będzie w stanie obsłużyć rosnącą liczbę użytkowników oraz operacji w jednostce czasu. Zrozumienie różnic oraz synergii między tymi pojęciami jest kluczowe dla każdego architekta oprogramowania¹.

Podczas gdy wydajność skupia się na optymalizacji istniejących zasobów w celu jak najszybszego przetwarzania pojedynczych żądań, skalowanie i replikacja odpowiadają na wyzwania związane z ograniczeniami sprzętowymi oraz koniecznością zapewnienia ciągłości działania systemu w przypadku awarii.

⁷ Grafana Documentation, *Dashboards overview* oraz *Grafana Alerting*, <https://grafana.com/docs/grafana/latest/fundamentals/dashboards-overview/>

¹ "Projektowanie systemów rozproszonych" – Podstawy skalowalności baz danych.

Wydajność baz danych

Wydajność (Performance) odnosi się do szybkości, z jaką system bazodanowy reaguje na zapytania i transakcje. Na poziomie pojedynczej instancji kluczowe znaczenie ma optymalizacja kodu SQL, poprawne projektowanie indeksów oraz unikanie kosztownych operacji, takich jak pełne skanowanie tabel (Full Table Scan). Optymalizator bazy danych stara się dobrać najlepszą ścieżkę wykonania zapytania, jednak jego skuteczność zależy od aktualności statystyk oraz struktury danych².

Po stronie sprzętowej wąskimi gardłami najczęściej stają się operacje wejścia/wyjścia (I/O) dysków, dostępna pamięć RAM (służąca jako bufor dla często odczytywanych stron danych) oraz moc procesora. Wyczerpanie tych zasobów bezpośrednio przekłada się na wzrost opóźnień (latency) i spadek przepustowości (throughput).

Skalowanie: Pionowe vs. Poziome

Gdy optymalizacja zapytań przestaje wystarczać, system musi zostać poddany skalowaniu. Skalowanie pionowe (Scaling Up) polega na zwiększaniu zasobów pojedynczej maszyny – dodaniu szybszych procesorów, większej ilości pamięci RAM lub wydajniejszych dysków SSD. Jest to najprostsze podejście, niepoprawiające jednak architektury aplikacji, ale ma swoje fizyczne i ekonomiczne granice (prawo malejących przychodów oraz limit techniczny serwera)³.

Skalowanie poziome (Scaling Out) polega na dodawaniu kolejnych maszyn do klastra. W kontekście baz danych jest to zadanie znacznie trudniejsze niż w przypadku bezstanowych aplikacji webowych, ponieważ wymaga mechanizmów dystrybucji danych. Jedną z głównych metod skalowania poziomego zapisu jest sharding, czyli fizyczny podział bazy na niezależne węzły.

Replikacja danych i wysoka dostępność

Replikacja to proces kopiowania danych z jednego serwera bazy danych (węzła głównego/Primary) na jeden lub więcej serwerów pomocniczych (węzłów potomnych/Replica). Głównym celem replikacji jest zapewnienie wysokiej dostępności (High Availability) – w razie awarii serwera głównego, jedna z replik może przejąć jego funkcję, minimalizując czas przestoju systemu⁴.

Dodatkowo replikacja pozwala na skalowanie operacji odczytu (Read Scalability). Aplikacja może kierować zapytania modyfikujące dane (INSERT, UPDATE) do węzła Primary, natomiast ciężkie zapytania raportowe i odczyty rozpraszać na repliki. Replikacja może odbywać się synchronicznie (gwarantuje spójność danych, ale zwalnia zapisy) lub asynchronicznie (szybsza, ale niesie ryzyko utraty ostatnich transakcji w przypadku nagłej awarii).

Wyzwania i kompromisy architektoniczne

Budowanie systemów wysoce skalowalnych i zreplikowanych wiąże się z koniecznością akceptacji kompromisów, co formalnie opisuje twierdzenie CAP (Consistency, Availability, Partition Tolerance). Mówi ono, że w rozproszonym systemie komputerowym można jednocześnie zapewnić tylko dwie z trzech wymienionych cech⁵.

W praktyce systemy bazodanowe często muszą wybierać pomiędzy silną spójnością danych (wszyscy użytkownicy widzą dokładnie to samo w tym samym momencie) a wysoką dostępnością i niskimi opóźnieniami, co doprowadziło do popularyzacji modeli spójności ostatecznej (Eventual Consistency) w bazach typu NoSQL.

Podsumowanie

Wydajność, skalowanie i replikacja nie są niezależnymi wyspami – to naczynia połączone. Dobrze zoptymalizowana pod kątem wydajności baza danych odwleka w czasie moment, w którym konieczne stanie się kosztowne skalowanie. Z kolei właściwie wdrożona replikacja nie tylko zabezpiecza system przed utratą danych, ale stanowi naturalny krok w kierunku poziomego skalowania odczytów. Architektura systemu musi być stale dostosowywana do dynamicznie zmieniających się wymagań biznesowych oraz skali operacji.

² "Optymalizacja zapytań i indeksowanie w relacyjnych bazach danych".

³ Analiza kosztów i barier technologicznych w skalowaniu pionowym.

⁴ Mechanizmy replikacji synchronicznej i asynchronicznej w systemach High Availability.

⁵ Twierdzenie CAP a praktyczne implementacje nowoczesnych magazynów danych.

2.2.6 Partycjonowanie danych

2.2.7 Bezpieczeństwo Baz Danych

Dział 7 przeglądu literaturowego poświęcony architekturze bezpieczeństwa, mechanizmom ochrony oraz analizie podatności w nowoczesnych systemach zarządzania bazami danych.

Autorzy

1. Oskar Mąka
2. Daniel Szokało
3. Dawid Szokało

Kierunek

Informatyka Techniczna, Politechnika Wrocławska

Przedmiot

Bazy Danych / Projekt Laboratoryjny

1. Model bezpieczeństwa CIA i ochrona infrastruktury

Rozważając bezpieczeństwo systemów bazodanowych, należy spojrzeć na problem warstwowo. Najniższą, a zarazem najbardziej fundamentalną warstwą jest ochrona samej infrastruktury sprzętowej i sieciowej, na której operuje silnik bazy danych (DBMS). Zanim przejdziemy do szczegółowych konfiguracji silnika, konieczne jest zdefiniowanie nadrzędnych celów bezpieczeństwa oraz zapewnienie izolacji środowiska uruchomieniowego.

Triada CIA w środowisku bazodanowym

Model **CIA** (ang. *Confidentiality, Integrity, Availability*) to klasyczny paradygmat bezpieczeństwa informacji. W kontekście relacyjnych i nierelacyjnych baz danych każdy z tych elementów realizowany jest przez specyficzne mechanizmy:

- **Poufność (Confidentiality):** Gwarantuje, że dane są dostępne wyłącznie dla autoryzowanych podmiotów. W bazach danych osiąga się to poprzez ścisłą kontrolę dostępu (np. RBAC), szyfrowanie danych w spoczynku (TDE - *Transparent Data Encryption*), maskowanie danych wrażliwych oraz szyfrowanie połączeń sieciowych (TLS).
- **Integralność (Integrity):** Zapewnia spójność i poprawność danych, chroniąc je przed nieautoryzowaną modyfikacją. Silniki bazodanowe realizują ten postulat natywnie poprzez właściwości ACID (szczególnie *Consistency*), więzy integralności (klucze obce, ograniczenia *CHECK*), a z punktu widzenia bezpieczeństwa – poprzez audytowanie DML (Data Manipulation Language) i mechanizmy wykrywania anomalii.
- **Dostępność (Availability):** Oznacza pewność, że system odpowie na żądanie w akceptowalnym czasie. Zagrożeniem dla dostępności są awarie sprzętowe oraz ataki DoS. Obroną na poziomie bazy danych są klastry wysokiej dostępności (HA), replikacja (np. *Master-Slave, Multi-Master*), partycjonowanie oraz odpowiednio zaprojektowane mechanizmy Disaster Recovery.

Izolacja sieciowa i architektura chmurowa (VPC)

Nawet najlepiej skonfigurowany system uprawnień bazodanowych traci na znaczeniu, jeśli instancja wystawiona jest bezpośrednio do publicznej sieci Internet. Standardem inżynieryjnym w nowoczesnych wdrożeniach (zarówno on-premise, jak i cloud) jest głęboka separacja sieciowa.

W środowiskach chmurowych (np. AWS, Azure, GCP) bazy danych umieszcza się w dedykowanych chmurach prywatnych (**VPC** - *Virtual Private Cloud*). Dobrą praktyką jest wdrożenie architektury wielowarstwowej:

1. **Warstwa publiczna (Public Subnet):** Zawiera load balancery i serwery webowe (np. Nginx), do których dostęp ma użytkownik końcowy.
2. **Warstwa aplikacji (Private Subnet):** Środowisko uruchomieniowe backendu (np. Node.js, Spring Boot).
3. **Warstwa danych (Database Subnet):** Najgłębiej ukryta podsieć prywatna bez routingu do Internetu (brak *Internet Gateway*). Dostęp do niej ma wyłącznie warstwa aplikacji poprzez ściśle określone reguły *Security Groups* lub *Network ACLs* (dopuszczające ruch tylko na konkretnym porcie, np. 5432 dla PostgreSQL, i tylko ze znanych adresów IP warstwy aplikacyjnej).

Takie podejście drastycznie zmniejsza powierzchnię ataku (ang. *attack surface*), eliminując ryzyko bezpośrednich ataków typu *brute-force* na porty bazy danych z zewnątrz.

Konteneryzacja a bezpieczeństwo bazy

Uruchamianie baz danych w kontenerach (np. Docker, Kubernetes) stało się powszechne, jednak niesie ze sobą specyficzne wektory zagrożeń. Kontenery dzielą jądro (kernel) z systemem hosta, co oznacza, że kompromitacja bazy danych może prowadzić do ucieczki z kontenera (ang. *container breakout*).

Aby zminimalizować to ryzyko, stosuje się następujące praktyki:

- **Brak uprawnień roota:** Procesy DBMS (np. `postgres` lub `mysqld`) nigdy nie powinny działać jako użytkownik `root` wewnątrz kontenera. Standardowe obrazy często wymagają jawnego zdefiniowania dyrektywy `USER` w pliku `Dockerfile`.
- **Ograniczenia zasobów (Cgroups):** Podatność silnika bazy na ataki typu *Denial of Service* można złagodzić na poziomie infrastruktury, nakładając twarde limity na zużycie CPU i pamięci RAM przez dany kontener. Zapobiega to sytuacji, w której przeciążona baza kładzie cały serwer hosta (Resource Exhaustion).
- **Read-Only Root Filesystem:** Kontener bazy danych powinien mieć zamontowany system plików w trybie tylko do odczytu, z wyjątkiem ściśle zdefiniowanych wolumenów (ang. *volumes*) przeznaczonych na pliki danych (np. `/var/lib/postgresql/data`). Uniemożliwia to atakującemu wstrzyknięcie złośliwych skryptów do katalogów systemowych po udanym wykorzystaniu luki aplikacyjnej.

2. Ataki na bazy danych i luki w zabezpieczeniach

Ponieważ bazy danych przechowują najbardziej krytyczne informacje organizacji, stanowią główny cel cyberataków. Według zestawień takich jak OWASP Top 10, podatności związane ze wstrzykiwaniem kodu (Injection) oraz błędną konfiguracją od lat utrzymują się w czołówce zagrożeń. Zrozumienie mechaniki tych ataków jest kluczowe do zaprojektowania skutecznych mechanizmów obronnych w warstwie aplikacyjnej i bazodanowej.

Wstrzykiwanie kodu SQL (SQL Injection)

SQL Injection (SQLi) to podatność polegająca na możliwości modyfikacji zapytania SQL poprzez nieodpowiednio zwalidowane dane wejściowe. Pozwala to atakującemu na wykonanie nieautoryzowanych operacji na bazie danych. Klasyczny przykład podatności (język PHP):

```
$username = $_POST['username'];
$query = "SELECT * FROM users WHERE username = '$username'";
$db->execute($query);
```

Jeśli atakujący jako parametr `username` przekaże ciąg `' OR '1'='1'`, zapytanie przyjmie postać:

```
SELECT * FROM users WHERE username = '' OR '1'='1';
```

Ponieważ warunek `'1'='1'` jest zawsze prawdziwy, silnik bazy danych zwróci wszystkie rekordy z tabeli `users`, omijając mechanizm uwierzytelniania.

Rodzaje ataków SQLi:

1. **In-Band SQLi (Classic):** Atakujący używa tego samego kanału komunikacyjnego do przeprowadzenia ataku i zebrania wyników. Najpopularniejsze to *Error-based SQLi* (wymuszanie błędów silnika w celu wycieku metadanych, np. struktury tabel) oraz *Union-based SQLi* (użycie operatora `UNION` do dołączenia złośliwych zapytań do oryginalnego).
2. **Inferential (Blind) SQLi:** Występuje, gdy aplikacja nie zwraca komunikatów o błędach ani bezpośrednich wyników zapytań. Atakujący musi wnioskować o strukturze danych na podstawie zachowania aplikacji.
 - * **Boolean-based:** Wysyłanie zapytań warunkowych i obserwowanie, czy strona ładuje się inaczej.
 - * **Time-based:** Zmuszanie bazy do wstrzymania działania (np. funkcjami `pg_sleep()` w PostgreSQL lub `WAITFOR DELAY` w SQL Server), aby potwierdzić obecność podatności.
3. **Out-of-Band SQLi:** Wykorzystywane rzadziej, opiera się na wymuszeniu przez silnik bazy danych żądania HTTP lub DNS do zewnętrznego serwera kontrolowanego przez atakującego.

Mitygacja: Podstawową i najskuteczniejszą metodą obrony przed SQLi jest korzystanie z **zapytań parametryzowanych (Prepared Statements)** lub nowoczesnych systemów ORM (Object-Relational Mapping), które oddzielają składnię zapytań od danych wejściowych.

Ataki NoSQL Injection

Wraz ze wzrostem popularności nierelacyjnych baz danych (np. MongoDB, CouchDB), pojawił się mit o ich całkowitej odporności na wstrzykiwanie kodu. Choć nie używają one tradycyjnego dialektu SQL, aplikacje z nich korzystające nadal mogą być podatne na modyfikację logiki zapytań.

Ataki te często bazują na wstrzykiwaniu operatorów bazy danych do zapytań, które aplikacja interpretuje jako obiekty JSON.

Przykład dla MongoDB: Aplikacja Node.js oczekuje poświadczeń przekazanych w formacie JSON:

```
db.collection('users').find({
  username: req.body.username,
  password: req.body.password
});
```

Atakujący wysyła zmodyfikowany payload z użyciem operatora logicznego \$ne (Not Equal):

```
{
  "username": {"$ne": null},
  "password": {"$ne": null}
}
```

W rezultacie silnik MongoDB wykonuje zapytanie szukające użytkownika, którego login i hasło *nie są nullem*. Zapytanie zwraca pierwszy pasujący rekord (często administratora), umożliwiając logowanie bez znajomości hasła.

Przepełnienie bufora (Buffer Overflow) w DBMS

Silniki baz danych (np. MySQL, PostgreSQL, Oracle) to skomplikowane aplikacje napisane najczęściej w językach C lub C++, co oznacza, że są potencjalnie narażone na błędy zarządzania pamięcią.

Buffer Overflow w kontekście baz danych występuje, gdy atakujący przesyła nieproporcjonalnie duży ciąg danych do bufora o stałej wielkości (np. w parametrze funkcji wbudowanej, mechanizmie autoryzacyjnym lub procedurze składowanej). Gdy dane wykraczają poza zaalokowany obszar pamięci, mogą nadpisać sąsiadujące obszary, w tym wskaźnik powrotu instrukcji (Instruction Pointer).

Może to prowadzić do: 1. **Awarji silnika bazy danych (Crash)** – przerywając ciągłość działania. 2. **Wykonania dowolnego kodu (RCE - Remote Code Execution)** – uruchomienia shellcode'u na serwerze z uprawnieniami procesu bazy danych, co stanowi bezpośrednie wejście do kompromitacji hosta (powiązane z ryzykiem opisanym w rozdziale o konteneryzacji).

Odmowa Usługi (DoS) na poziomie bazy danych

Ataki Denial of Service na bazy danych różnią się od klasycznych ataków sieciowych. Mają one charakter aplikacyjny i celują w wyczerpanie zasobów sprzętowych serwera (CPU, RAM, I/O) lub limitów samego oprogramowania.

- **Wyczerpanie puli połączeń (Connection Exhaustion):** Każdy DBMS ma zdefiniowany limit maksymalnej liczby jednoczesnych połączeń (np. parametr `max_connections` w PostgreSQL). Atakujący, często wykorzystując luki w warstwie aplikacji, może zainicjować tysiące zawieszonych lub bardzo powolnych sesji bazy, uniemożliwiając logowanie legalnym użytkownikom.
- **Asymetryczne obciążenie (Query-based DoS):** Atakujący wstrzykuje i uruchamia zapytania, które są syntaktycznie poprawne, ale drastycznie nieoptymalne kosztowo. Przykładem może być wymuszenie iloczynu kartezjańskiego (CROSS JOIN) na bardzo dużych tabelach bez odpowiednich indeksów, co może zablokować procesor serwera na kilka minut i doprowadzić do blokady całej aplikacji (Deadlock/Timeouts).

Automatyzacja ataków: Narzędzia audytowe

W procesach testów penetracyjnych baz danych, analitycy bezpieczeństwa korzystają ze zautomatyzowanych frameworków. Najpopularniejszym z nich jest **sqlmap** – potężne narzędzie open-source automatyzujące proces wykrywania podatności Sqli i przejmowania kontroli nad serwerami bazodanowymi.

W typowym scenariuszu audytu, uruchomienie polecenia:

```
sqlmap -u "http://podatna-aplikacja.local/item.php?id=1" --dbs
```

pozwoli narzędziu na wykrycie typu wstrzyknięcia (np. czasowe lub błędowe), zidentyfikowanie używanego silnika DBMS oraz – w razie sukcesu – wyliczenie wszystkich baz danych dostępnych na serwerze (enumeracja instancji).

3. Zarządzanie tożsamością, uwierzytelnianie i autoryzacja

Po zabezpieczeniu infrastruktury sieciowej oraz aplikacji przed atakami typu Injection, kolejną linią obrony jest ścisła kontrola dostępu do samych danych. W architekturze systemów bazodanowych proces ten opiera się na trzech filarach: zarządzaniu tożsamością (Identity Management), uwierzytelnianiu (Authentication) oraz autoryzacji (Authorization). Prawidłowa implementacja tych mechanizmów gwarantuje, że zasada najmniejszego uprzywilejowania (PoLP - Principle of Least Privilege) może być skutecznie egzekwowana w całym systemie.

Uwierzytelnianie w bazach danych

Proces uwierzytelniania weryfikuje, czy podmiot (użytkownik końcowy, administrator lub usługa aplikacyjna) łączący się z bazą danych jest tym, za kogo się podaje. Współczesne silniki relacyjne (np. PostgreSQL, Oracle) oraz nierelacyjne odeszły od prostego przesyłania haseł tekstem jawnym (plaintext), wprowadzając bardziej zaawansowane mechanizmy:

- **Kryptograficzne protokoły uwierzytelniania:** Zamiast przestarzałych algorytmów (np. MD5), nowoczesne wdrożenia wykorzystują mechanizmy typu SCRAM-SHA-256 (Salted Challenge Response Authentication Mechanism). SCRAM zapobiega atakom typu *replay attack* oraz minimalizuje skutki ewentualnej kradzieży skrótów z serwera, ponieważ hasło nigdy nie jest przesyłane w otwartej formie.
- **Integracja z dostawcami tożsamości (IdP):** W środowiskach korporacyjnych unika się tworzenia lokalnych kont wewnątrz bazy (tzw. *database-native accounts*) dla użytkowników fizycznych. Zamiast tego silniki DBMS integruje się z centralnymi usługami katalogowymi (LDAP, Active Directory) lub wykorzystuje protokół Kerberos (np. poprzez GSSAPI). Pozwala to na centralne zarządzanie cyklem życia konta i automatyczne odbieranie dostępu w przypadku odejścia pracownika (tzw. *offboarding*).
- **Uwierzytelnianie certyfikatowe (mTLS - Mutual TLS):** Standard powszechnie stosowany w architekturze mikroserwisowej do komunikacji *machine-to-machine*. Oprócz szyfrowania samego kanału (TLS), serwer bazy danych żąda i weryfikuje certyfikat klienta (X.509) wystawiony przez wewnętrzne, zaufane centrum certyfikacji (CA).

Autoryzacja i modele kontroli dostępu

Po udanym uwierzytelnieniu następuje faza autoryzacji, czyli weryfikacji, jakie operacje (SELECT, INSERT, UPDATE, DELETE) i na jakich obiektach (tabele, widoki, procedury) dany podmiot może wykonać.

Kontrola dostępu oparta na rolach (RBAC - Role-Based Access Control)

Model RBAC to absolutny standard w systemach zarządzania bazami danych. Zamiast nadawać uprawnienia (GRANT) bezpośrednio pojedynczym użytkownikom, tworzy się logiczne role odzwierciedlające funkcje biznesowe lub techniczne (np. `app_readonly`, `app_writer`, `dba_admin`). Następnie konta personalne lub serwisowe przypisuje się do tych ról. Taka abstrakcja drastycznie upraszcza audyt i zarządzanie uprawnieniami.

Przykład implementacji RBAC (dialekt PostgreSQL):

```
-- Utworzenie technicznej roli tylko do odczytu
CREATE ROLE app_readonly;
GRANT CONNECT ON DATABASE production TO app_readonly;
GRANT USAGE ON SCHEMA public TO app_readonly;
```

(continues on next page)

(continued from previous page)

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_readonly;

-- Przypisanie użytkownika raportującego do przygotowanej roli
GRANT app_readonly TO report_user;
```

Bezpieczeństwo na poziomie wierszy (Row-Level Security)

Tradycyjny model autoryzacji w bazach danych działa na poziomie całych struktur (np. blokuje lub zezwala na odczyt całej tabeli). Współczesne aplikacje, zwłaszcza budowane w modelu wielodostępnym (multi-tenant), często wymagają znacznie bardziej granularnej kontroli, aby użytkownik jednego klienta nie miał fizycznej możliwości odczytania rekordów należących do innego.

Z pomocą przychodzi mechanizm Row-Level Security (RLS). Pozwala on na nakładanie polityk bezpieczeństwa bezpośrednio na wiersze danych. Silnik bazy automatycznie i w sposób przezroczysty dokleja zdefiniowane warunki do każdego zapytania. Kluczową zaletą RLS jest fakt, że uniemożliwia on ominięcie logiki izolacyjnej w warstwie aplikacji – nawet w przypadku wystąpienia luki SQL Injection.

Przykład definicji RLS (dialekt PostgreSQL):

```
-- Włączenie weryfikacji RLS dla krytycznej tabeli
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;

-- Utworzenie polityki izolującej dane między użytkownikami
CREATE POLICY isolate_tenant_policy ON orders
    USING (tenant_id = current_setting('app.current_tenant')::int);
```

Zarządzanie poświadczeniami aplikacji (Secrets Management)

Nawet najbardziej rygorystyczne mechanizmy wewnątrz samej bazy danych zawiodą, jeśli poświadczenia do niej (tzw. *connection strings*) zostaną umieszczone na stałe (hardcoded) w kodzie źródłowym i wypchnięte do repozytorium kodu.

Zgodnie z dobrymi praktykami nurtu DevSecOps, środowiska produkcyjne powinny wykorzystywać zewnętrzne systemy zarządzania sekretami (np. HashiCorp Vault, AWS Secrets Manager). Narzędzia te pozwalają na dynamiczne generowanie tymczasowych, krótkożyjących poświadczeń (tzw. *dynamic secrets*). W takim scenariuszu aplikacja uwierzytelnia się do Vaulta i otrzymuje login oraz hasło do bazy danych, które wygasają np. po godzinie. Po tym czasie usługa zarządzająca automatycznie usuwa (REVOKE) te poświadczenia z silnika bazy, co radykalnie zmniejsza okno potencjalnego ataku w przypadku ich wycieku.

4. Kryptografia w bazach danych

Nawet najbardziej rygorystyczne mechanizmy kontroli dostępu i izolacji sieciowej mogą zawieść w obliczu zaawansowanych ataków (tzw. APT – Advanced Persistent Threats) lub błędów ludzkich. W myśl zasady obrony w głąb (Defense in Depth), ostatnią linią ochrony informacji jest kryptografia. Jej implementacja w środowiskach bazodanowych dzieli się na trzy główne obszary: ochronę danych w spoczynku, w tranzycie oraz w użyciu.

Szyfrowanie danych w spoczynku (Data at Rest)

Szyfrowanie w spoczynku ma na celu zabezpieczenie fizycznych nośników danych przed nieautoryzowanym odczytem, na przykład w przypadku kradzieży serwera, dysku twardego lub nieautoryzowanego skopiowania plików kopii zapasowych.

Przemysłowym standardem w tym zakresie jest mechanizm **TDE (Transparent Data Encryption)**, natywnie wspierany przez silniki takie jak Microsoft SQL Server czy Oracle (dla PostgreSQL stosuje się często szyfrowanie na poziomie systemu plików, np. LUKS, lub dedykowane rozszerzenia). TDE działa na poziomie warstwy wejścia/wyjścia (I/O) silnika bazy danych:

- **Przezroczystość:** Aplikacja kliencka nie musi być modyfikowana. Dane są szyfrowane tuż przed zapisem na dysk i deszyfrowane w momencie wczytywania ich do pamięci operacyjnej (RAM) serwera.
- **Architektura kluczy:** Bezpieczeństwo TDE opiera się na hierarchii kluczy kryptograficznych. Dane właściwe szyfrowane są kluczem symetrycznym DEK (Data Encryption Key). Sam DEK jest z kolei chroniony (szyfrowany) asymetrycznym kluczem nadrzędnym MEK (Master Encryption Key), który powinien być przechowywany poza samym serwerem bazy danych – najczęściej w sprzętowych modułach bezpieczeństwa (HSM) lub chmurowych usługach KMS (Key Management Service).

Szyfrowanie danych w tranzycie (Data in Transit)

Przesyłanie danych między warstwą aplikacji a silnikiem bazy danych w postaci jawnej (plaintext) naraża system na ataki typu Man-in-the-Middle (MitM) oraz podsłuchiwanie pakietów (sniffing).

Aby temu zapobiec, wszystkie połączenia z bazą danych muszą być obligatoryjnie szyfrowane za pomocą protokołu **TLS (Transport Layer Security)**. Wymuszenie bezpiecznego kanału realizuje się zazwyczaj poprzez odpowiednią konfigurację parametrów połączenia.

Przykład wymuszenia TLS w konfiguracji PostgreSQL (`postgresql.conf`):

```
# Włączenie obsługi protokołu TLS
ssl = on
ssl_cert_file = '/etc/ssl/certs/db-server.crt'
ssl_key_file = '/etc/ssl/private/db-server.key'
# Odrzucanie połączeń używających przestarzałych, podatnych wersji protokołu
ssl_min_protocol_version = 'TLSv1.2'
```

Po stronie klienta, ciąg połączeniowy (connection string) powinien zawierać dyrektywę bezwzględnie wymagającą weryfikacji certyfikatu (np. parametr `sslmode=verify-full`).

Szyfrowanie na poziomie klienta (Client-Side Encryption)

Mechanizm TDE chroni przed fizyczną kradzieżą nośników, ale nie zabezpiecza przed administratorem bazy danych (DBA) lub atakiem SQL Injection, ponieważ w momencie działania zapytania dane w pamięci serwera są w pełni odszyfrowane.

Odpowiedzią na ten problem jest szyfrowanie na poziomie aplikacji (lub kolumn w nowszych silnikach, np. funkcja *Always Encrypted* w SQL Server). W tym paradygmacie dane są szyfrowane w warstwie aplikacji (np. backendzie w Javie lub C#) jeszcze przed wysłaniem ich do bazy.

Silnik bazy danych operuje wyłącznie na kryptogramach (ciphertext) i nigdy nie ma dostępu do kluczy deszyfrujących, co zapewnia najwyższy poziom poufności dla najbardziej wrażliwych kolumn (np. numerów PESEL, danych kart kredytowych). Kosztem takiego rozwiązania jest utrata możliwości wykonywania zaawansowanych operacji analitycznych (takich jak sumowanie, sortowanie czy wyszukiwanie z użyciem operatora LIKE) bezpośrednio po stronie DBMS.

Dynamiczne maskowanie danych i funkcje skrótu

Uzupełnieniem kryptografii jest technika **DDM (Dynamic Data Masking)**. Służy ona do ochrony wrażliwych danych w czasie rzeczywistym przed użytkownikami, którzy nie posiadają odpowiednich poświadczeń (np. analitykami biznesowymi lub programistami korzystającymi z kopii produkcyjnej). DDM maskuje wyniki zapytań (np. zwracając ciąg `****_****_****-1234` zamiast pełnego numeru karty), podczas gdy same dane na dysku pozostają nienaruszone.

Ponadto, wracając do zagadnień z zakresu uwierzytelniania, należy pamiętać o kryptografii jednokierunkowej. Wrażliwe poświadczenia (takie jak hasła użytkowników aplikacji) nigdy nie mogą być przechowywane w formie szyfrowalnej (dwukierunkowej). Zamiast tego muszą być poddawane procesowi solenia (salting) i hashowania z wykorzystaniem funkcji spowalniających (Key Derivation Functions), takich jak **Argon2**, **bcrypt** lub **PBKDF2**, co skutecznie uniemożliwia ataki słownikowe i wykorzystanie tęczyowych tablic (rainbow tables).

5. Audyt, logowanie i wykrywanie anomalii

Nawet najsilniejsze mechanizmy prewencyjne, takie jak kontrola dostępu czy szyfrowanie, nie gwarantują stuprocentowego bezpieczeństwa. W przypadku udanego ataku lub błędu wewnętrznego (tzw. zagrożenia *Insider Threat*), kluczowe staje się odtworzenie przebiegu zdarzeń. Temu celowi służą mechanizmy audytu i logowania, które stanowią fundament dla systemów wykrywania anomalii oraz zapewniają niezaprzeczalność (ang. *Non-repudiation*) operacji wykonywanych na danych.

Znaczenie audytu i rodzaje logów bazodanowych

Audytowanie bazy danych to proces polegający na monitorowaniu i rejestrowaniu akcji użytkowników oraz procesów systemowych. Nie należy mylić go z klasycznymi logami transakcyjnymi (np. WAL w PostgreSQL czy Redo Logs w Oracle), które służą do odtwarzania bazy po awarii. Audyt bezpieczeństwa skupia się na tym **kto**, **kiedy**, **skąd** i **jaka** operację wykonał.

Większość nowoczesnych silników DBMS pozwala na granularną konfigurację logowania. Rejestrowane zdarzenia można podzielić na kilka kategorii: * **Logi uwierzytelniania**: Rejestrują udane i nieudane próby logowania (np. błędy *Access Denied*), co pozwala na szybkie wykrycie ataków typu brute-force. * **Logi DDL (Data Definition Language)**: Śledzą zmiany w strukturze bazy (tworzenie i usuwanie tabel, modyfikacje uprawnień). * **Logi DML (Data Manipulation Language)**: Najbardziej kosztowne wydajnościowo, ale niezbędne w systemach o wysokim rygorze bezpieczeństwa (np. bankowość, opieka zdrowotna). Pozwalają na śledzenie zapytań SELECT, UPDATE czy DELETE na wrażliwych tabelach.

Przykładowo, natywne logowanie w PostgreSQL jest często niewystarczające dla zaawansowanego audytu. W środowiskach produkcyjnych powszechnie stosuje się rozszerzenie **pgAudit** (PostgreSQL Audit Extension), które dostarcza szczegółowe logowanie sesji i obiektów przy użyciu standardowych mechanizmów logowania wbudowanych w silnik.

Centralizacja logów i systemy SIEM

Przechowywanie logów audytowych wyłącznie na tym samym serwerze, na którym działa baza danych, jest poważnym błędem architektonicznym. W przypadku udanej kompromitacji hosta (np. poprzez ucieczkę z kontenera opisaną w rozdziale 1), atakujący w pierwszej kolejności zacierza ślady, modyfikując lub usuwając pliki logów.

Dlatego standardem rynkowym jest natychmiastowy eksport logów w czasie rzeczywistym do niezależnych systemów centralnych. Wykorzystuje się do tego architekturę SIEM (ang. *Security Information and Event Management*), z wykorzystaniem stosów technologicznych takich jak **ELK** (Elasticsearch, Logstash, Kibana) lub **Splunk**. Systemy te agregują logi nie tylko z baz danych, ale również z warstwy aplikacji, firewalli i systemów operacyjnych, pozwalając na korelację zdarzeń z różnych źródeł w celu identyfikacji złożonych ataków.

Database Activity Monitoring (DAM) i wykrywanie anomalii

Tradycyjna analiza logów jest procesem reaktywnym (po fakcie). Aby przejść na model proaktywny, stosuje się systemy DAM (ang. *Database Activity Monitoring*), które monitorują ruch do bazy danych z zewnątrz (np. analizując ruch sieciowy (tzw. *packet sniffing*) lub podpinając się pod pule pamięci silnika). Dzięki temu system DAM jest odseparowany od samej bazy, co zapobiega modyfikacji jego działania nawet po przejęciu uprawnień administratora bazy (DBA).

Kluczową funkcją nowoczesnych systemów z tej kategorii jest **wykrywanie anomalii w oparciu o analizę behawioralną (UEBA)**. Zamiast opierać się wyłącznie na statycznych sygnaturach zagrożeń, systemy te wykorzystują algorytmy uczenia maszynowego do modelowania “normalnego” zachowania aplikacji i użytkowników.

System wykryje anomalię i podniesie alarm (lub zablokuje zapytanie), gdy np.: * Aplikacja webowa, która zazwyczaj pobiera pojedyncze rekordy w zapytaniach SELECT, nagle próbuje wykonać zrzut (ang. *dump*) tysięcy wierszy z tabeli klientów – co może świadczyć o udanym wstrzyknięciu kodu (SQLi z rozdziału 2) i próbie eksfiltracji danych. * Użytkownik o uprawnieniach administracyjnych loguje się do bazy z nietypowej geolokalizacji lub poza standardowymi godzinami pracy. * Gwałtownie rośnie liczba odrzucanych połączeń lub błędów składniowych SQL z określonego adresu IP wewnątrz sieci.

6. Bezpieczeństwo kopii zapasowych i Disaster Recovery

Systemy prewencyjne i detekcyjne nie zawsze są w stanie zatrzymać zaawansowane ataki, takie jak ransomware, czy też zapobiec fizycznym awariom infrastruktury. W takich scenariuszach ostateczną linią obrony stają się mechanizmy odtwarzania danych po awarii. Bezpieczeństwo samych kopii zapasowych jest równie krytyczne co bezpieczeństwo głównej instancji bazy danych – skompromitowany backup często ostatecznie przekreśla szanse na powrót organizacji do normalnego funkcjonowania operacyjnego.

Zasada 3-2-1 i niezmienność kopii (Immutable Backups)

Klasyczna reguła 3-2-1 (trzy kopie, na dwóch różnych nośnikach, z czego jedna w innej lokalizacji fizycznej lub geograficznej) pozostaje fundamentem ochrony danych. Jednakże, w dobie zaawansowanych ataków kryptograficznych, wymaga ona uzupełnienia o koncepcję niezmienności (ang. *Immutability*).

Współczesne złośliwe oprogramowanie często priorytetyzuje poszukiwanie dysków sieciowych oraz zasobów chmurowych z kopiami zapasowymi, aby je usunąć lub zaszyfrować przed zaatakowaniem głównego klastra bazy danych. Odpowiedzią inżynierską na to zagrożenie jest technologia zapisu WORM (ang. *Write Once, Read Many*). W nowoczesnej architekturze chmurowej (np. za pomocą mechanizmu AWS S3 Object Lock) gwarantuje ona, że zapisany plik backupu nie może zostać w żaden sposób zmodyfikowany ani usunięty przez zdefiniowany okres retencji. Ochrona ta działa nawet w przypadku całkowitej kompromitacji konta z najwyższymi uprawnieniami (ang. *root account compromise*).

Szyfrowanie archiwów i zarządzanie kluczami

Ponieważ logiczne i fizyczne kopie zapasowe opuszczają bezpieczne środowisko serwerowni lub głęboko izolowanej podsieci (VPC, o której mowa w rozdziale 1), absolutnym wymogiem jest ich szyfrowanie w spoczynku (ang. *Data at Rest Encryption*). Wykonanie zwykłego rzutu bazy w postaci czystego tekstu i przesłanie go na zewnątrz to drastyczne ryzyko wycieku, uderzające bezpośrednio w postulat poufności.

W nowoczesnych wdrożeniach archiwizacji stosuje się dedykowane systemy KMS (ang. *Key Management Service*). Klucze szyfrujące używane do ochrony backupów powinny być ściśle odseparowane od infrastruktury samego klastra bazodanowego. Dodatkowo podczas procesu odtwarzania konieczna jest weryfikacja integralności archiwów przy użyciu funkcji skrótu (np. SHA-256), co pozwala upewnić się, że paczka danych nie uległa uszkodzeniu na etapie przesyłania.

Disaster Recovery (DR) i wskaźniki RTO/RPO

Kopia zapasowa to jedynie nośnik danych; z perspektywy ciągłości biznesowej (ang. *Business Continuity*) liczy się sprawdzony proces ich przywracania. Architektura Disaster Recovery dla baz danych opiera się na dwóch krytycznych metrykach:

- **RPO (Recovery Point Objective):** Maksymalny akceptowalny czas, z którego dane mogą zostać bezpowrotnie utracone. Minimalizację tego wskaźnika do okolic zera osiąga się poprzez ciągłą strumieniową archiwizację logów transakcyjnych (np. mechanizm *WAL archiving* w PostgreSQL).
- **RTO (Recovery Time Objective):** Czas niezbędny na przywrócenie systemów bazodanowych do pełnego działania operacyjnego po awarii.

Dla systemów o rygorystycznym RTO (wymagających dostępności mierzonej w sekundach) tradycyjne odtwarzanie z plików jest niewystarczające. Stosuje się wówczas architekturę klastrów rozproszonych w modelach *Active-Passive* (ciągła, asynchroniczna replikacja do zapasowego centrum danych) lub *Active-Active* (synchroniczny zapis w wielu lokalizacjach, wymagający specjalnego rozwiązywania problemów ze spójnością i opóźnieniami sieciowymi, tzw. *split-brain*).

Niezależnie od stopnia skomplikowania architektury chmurowej, każda polityka Disaster Recovery pozostaje martwym dokumentem, dopóki nie zostanie poddana rygorystycznym, regularnym i udokumentowanym testom odtworzeniowym (*Disaster Recovery Drills*) w odizolowanym środowisku.

2.2.8 Kopie zapasowe i odzyskiwanie danych

Autorzy: Norbert Antonovitch, Michał Bednarczyk, Jan Balazs de Borbatviz

Wstęp

Kopie zapasowe w PostgreSQL można podzielić na dwa główne nurty: kopie logiczne wykonywane narzędziami `pg_dump` i `pg_dumpall` oraz kopie fizyczne całego klastra wykonywane m.in. przez `pg_basebackup` wraz z archiwizacją WAL dla odzyskiwania punktowego PITR (Point-in-Time Recovery). Kopie logiczne są wygodne do odtwarzania pojedynczych obiektów i pojedynczych baz, natomiast kopie fizyczne są podstawą pełnego odtworzenia instancji, tablespaces i odzyskiwania po awariach.

Tworzenie kopii zapasowej całego systemu PostgreSQL - mechanizmy wbudowane

Najprostszym wbudowanym sposobem wykonania kopii całego klastra jest `pg_dumpall`, które eksportuje wszystkie bazy w klastrze oraz obiekty globalne wspólne dla całej instancji, takie jak role i przestrzenie tabel. Taki zrzut ma postać tekstowego skryptu SQL i odtwarza się go zwykle przez `psql`, ale metoda ta jest logiczna, więc nie daje możliwości odzyskiwania punktowego przez WAL.

Drugim, ważniejszym z punktu widzenia odporności na awarie mechanizmem jest `pg_basebackup`, które tworzy fizyczną kopię działającego klastra bez blokowania normalnej pracy klientów. Narzędzie to wykonuje kopię całego klastra, a nie pojedynczych baz czy tabel, może pracować w formacie katalogowym lub `tar` i może dołączać wymagane pliki WAL metodą `stream`, `fetch` albo `none`. Dokumentacja podkreśla też, że taka kopia może służyć zarówno do PITR, jak i do zbudowania serwera standby.

Aby pełna kopia fizyczna była użyteczna w scenariuszu odtwarzania do wybranego momentu, należy włączyć archiwizację WAL przez ustawienie co najmniej `wal_level = replica`, `archive_mode = on` oraz poprawnego `archive_command` albo `archive_library`. PostgreSQL zapisuje każdą zmianę w logu WAL, a po odtworzeniu kopii bazowej można odtworzyć kolejne segmenty WAL, aby dojść do stanu bieżącego lub do wskazanego punktu w czasie.

Przykładowe polecenia:

```
# logiczna kopia całego klastra
pg_dumpall -U postgres --clean --file=cluster.sql

# fizyczna kopia całego klastra z dołączeniem WAL
pg_basebackup -h dbserver -D /backup/base -Fp -X stream -P
```

W praktyce do dokumentacji laboratoryjnej warto zaznaczyć, że `pg_dumpall` lepiej nadaje się do migracji i prostych kopii administracyjnych, a `pg_basebackup` z archiwizacją WAL do scenariuszy disaster recovery.

Tworzenie kopii zapasowych poszczególnych baz danych - mechanizmy wbudowane

Do wykonania kopii pojedynczej bazy służy `pg_dump`, które tworzy spójny zrzut nawet wtedy, gdy baza jest równocześnie używana przez innych użytkowników, i nie blokuje zwykłych operacji odczytu ani zapisu. Narzędzie obsługuje format tekstowy, `custom`, `directory` oraz `tar`, przy czym formaty archiwalne współpracują z `pg_restore` i pozwalają na selektywne odtwarzanie obiektów.

Istotne jest to, że `pg_dump` wykonuje kopię tylko jednej bazy danych. Format `custom` (`-Fc`) i `directory` (`-Fd`) są najbardziej elastyczne, bo pozwalają wybierać odtwarzane elementy, zmieniać kolejność odtwarzania, a w części scenariuszy także korzystać z odtwarzania równoległego.

Przykładowe polecenia:

```
# kopia pojedynczej bazy w formacie custom
pg_dump -U postgres -d labdb -Fc -f /backup/labdb.dump

# kopia pojedynczej bazy jako skrypt SQL
pg_dump -U postgres -d labdb -Fp -f /backup/labdb.sql
```

(continues on next page)

(continued from previous page)

```
# odtworzenie archiwum custom  
pg_restore -d labdb_restored /backup/labdb.dump
```

Jeżeli celem jest ochrona wybranych schematów lub tabel, `pg_dump` pozwala zawęzić zakres eksportu przez opcje takie jak `-n` dla schematu i `-t` dla tabeli. Trzeba jednak zaznaczyć, że wybiórczy zrzut nie gwarantuje kompletności zależności wszystkich obiektów w nowym, pustym środowisku, jeśli pominięto elementy, od których zależy odtwarzany obiekt.

Odzyskiwanie usuniętych lub uszkodzonych danych

Sposób odzyskiwania zależy od tego, czy problem dotyczy pojedynczego obiektu logicznego, całej bazy, przestrzeni tabel, czy fizycznego uszkodzenia danych. PostgreSQL rozróżnia w praktyce odzyskiwanie logiczne z plików dump oraz odzyskiwanie fizyczne z kopii bazowej i archiwum WAL.

Odtwarzanie tabel i innych obiektów logicznych

Jeżeli wcześniej wykonano kopię `pg_dump` w formacie archiwalnym, można odtworzyć pojedynczą tabelę lub inny obiekt selektywnie przy pomocy `pg_restore`, bez przywracania całej bazy. To jest najprostszy wariant odzyskania przypadkowo usuniętej tabeli, o ile odpowiedni obiekt znajdował się w logicznej kopii zapasowej.

Przykład:

```
pg_restore -d labdb -t public.wyniki /backup/labdb.dump
```

Odtwarzanie usuniętej bazy danych

Usuniętą bazę można najłatwiej odtworzyć z wcześniejszego zrzutu `pg_dump` tej konkretnej bazy albo z `pg_dumpall`, jeżeli wykonywano kopię całego klastra. W przypadku wymogu odtworzenia dokładnie do chwili sprzed usunięcia, potrzebna jest kopia fizyczna oraz archiwum WAL, ponieważ same zrzuty logiczne nie są częścią rozwiązywania continuous archiving i nie wspierają PITR.

Odtwarzanie przestrzeni tabel i pełnego klastra

Przestrzenie tabel są obiektami globalnymi klastra, dlatego ich definicje są uwzględniane przez `pg_dumpall`, a przy kopiach fizycznych `pg_basebackup` zachowuje także ich strukturę i mapowanie. Dokumentacja `pg_basebackup` wskazuje, że przy pracy z tablespaces można użyć mapowania `--tablespace-mapping`, a przy odtwarzaniu trzeba zweryfikować poprawność dowiązań symbolicznych i lokalizacji danych.

Przy fizycznym uszkodzeniu danych, katalogu PGDATA albo tablespaces standardowa procedura obejmuje odtworzenie kopii bazowej, usunięcie starych plików danych, odtworzenie katalogów danych i tablespaces, skonfigurowanie `restore_command`, utworzenie pliku `recovery.signal`, a następnie ponowne uruchomienie serwera w trybie recovery. PostgreSQL podczas odtwarzania odczytuje wymagane segmenty WAL i może dojść do stanu bieżącego albo zatrzymać się na wybranym punkcie odzyskiwania.

PITR i odzyskiwanie stanu sprzed błędu

Najważniejszym mechanizmem odzyskiwania po usunięciu danych operacyjnych jest PITR, czyli przywrócenie systemu do chwili sprzed błędnej operacji, na przykład `DROP TABLE` lub `DROP DATABASE`. W takim scenariuszu odtwarza się kopię bazową, konfiguruje `restore_command`, a następnie ustawia cel odzyskiwania, np. przez czas, nazwany punkt przywracania lub identyfikator transakcji.

To rozwiązanie ma jednak ważne ograniczenie: nie służy do odzyskania pojedynczej tabeli “w miejscu” bez wpływu na resztę systemu, lecz do odtworzenia całego klastra do wcześniejszego stanu. Dlatego w praktyce laboratoryjnej często łączy się oba podejścia: regularne `pg_dump` dla obiektów logicznych i `pg_basebackup` z archiwizacją WAL dla pełnego recovery.

Dedykowane oprogramowanie i skrypty zewnętrzne do automatyzacji

Wbudowane narzędzia PostgreSQL są wystarczające do ręcznego wykonywania kopii, ale w środowisku produkcyjnym często wykorzystuje się oprogramowanie automatyzujące harmonogramy, retencję, walidację i odtwarzanie. Dwa najczęściej wskazywane rozwiązania to Barman i pgBackRest, przy czym dostępna dokumentacja Barman bezpośrednio opisuje obsługę pełnych i przyrostowych kopii, archiwizacji WAL oraz automatycznego uruchamiania backupów.

Barman wykonuje kopie całego serwera PostgreSQL i wymaga poprawnej archiwizacji WAL, oferując różne metody kopii, w tym streaming przez `pg_basebackup`, `rsync` oraz backupy snapshotowe w chmurze. Dokumentacja podaje też wsparcie dla backupów przyrostowych, limitowania pasma, kompresji i pracy z harmonogramem przez zewnętrzne mechanizmy systemowe, np. `cron`.

Przykładowe zastosowania narzędzi zewnętrznych:

- **Barman:** centralne repozytorium backupów, archiwizacja WAL, backupy pełne i przyrostowe, odzyskiwanie całych instancji.
- **pgBackRest:** popularne narzędzie do wydajnych backupów i retencji, często używane w środowiskach HA oraz większych instalacjach PostgreSQL.
- **Własne skrypty Bash/PowerShell:** wywołanie `pg_dump`, `pg_dumpall` lub `pg_basebackup`, kompresja plików, rotacja katalogów, logowanie i wysyłka wyników przez e-mail lub do systemu monitoringu.

Przykładowy prosty skrypt automatyzujący kopię jednej bazy:

```
#!/bin/bash
DATA=$(date +%F_%H-%M)
KATALOG=/backup/postgres
BAZA=labdb

mkdir -p "$KATALOG"
pg_dump -U postgres -d "$BAZA" -Fc -f "$KATALOG/${BAZA}_${DATA}.dump"
find "$KATALOG" -type f -name "${BAZA}_*.dump" -mtime +7 -delete
```

Taki skrypt jest prosty, ale nie zapewnia pełnego disaster recovery, bo nie obejmuje archiwizacji WAL i nie chroni całego klastra. Z tego powodu dla systemów krytycznych bardziej właściwe są narzędzia klasy Barman lub pgBackRest, które porządkują pełny proces backupu, retencji, testów i odtwarzania.

Podsumowanie

W rozdziale warto wyraźnie rozróżnić kopie logiczne i fizyczne, bo odpowiadają one na inne potrzeby eksploatacyjne. `pg_dump` i `pg_dumpall` są najlepsze do prostych eksportów i selektywnego odtwarzania, natomiast `pg_basebackup` z archiwizacją WAL jest podstawą odzyskiwania po poważnej awarii i realizacji PITR.

2.3 Rozdział 3: Projektowanie Bazy Danych: System Zarządzania Biblioteką

Autorzy

1. Paweł Łoćwin
2. Paweł Łosowski

2.3.1 Wybór zagadnienia, opis procesów i danych

Wybrane zagadnienie:

System zarządzania wypożyczeniami w bibliotece. Projekt obejmuje obsługę bazy czytelników, katalogu zbiorów bibliotecznych (z uwzględnieniem autorów i kategorii literackich), a także pełen proces ewidencji wypożyczeń, zwrotów oraz historii operacji każdego użytkownika.

Opis procesów i więzy integralności:

Głównym procesem w systemie jest obieg książki między biblioteką a czytelnikiem.

- **Rejestracja:** Czytelnik zapisuje się do biblioteki, podając swoje dane osobowe oraz adresowe. System automatycznie rejestruje datę zapisu.
- **Katalogowanie:** Książki są kategoryzowane (np. Fantastyka, Kryminał, Literatura faktu) i przypisane do konkretnych autorów. Każdy fizyczny egzemplarz książki posiada swój unikalny identyfikator, co umożliwia śledzenie niezależnie każdego egzemplarza.
- **Wypożyczenia i Zwroty:** Proces wypożyczenia łączy czytelnika z konkretnym egzemplarzem książki w czasie. Rejestrowana jest data wypożyczenia. System zakłada konieczność ewidencji dokładnej daty zwrotu rzeczywistego, co pozwala na analizy przeterminowanych wypożyczeń i wzorców korzystania z kolekcji.
- **Statystyki (Więzy integralności):** Pola takie jak “aktualne wypożyczenia” i “suma wypożyczeń” z pierwotnego prototypu są wartościami wyliczalnymi (pochodnymi) na podstawie historii transakcji i są usuwane ze schematu normalnego.

Wykaz gromadzonych danych:

- **Dane czytelnika:** Imię, Nazwisko, Ulica, Kod pocztowy, Miasto, Data zapisu.
- **Dane książki:** Tytuł, Rok wydania, Dostępność.
- **Dane autora:** Imię, Nazwisko.
- **Dane kategorii:** Nazwa gatunku literackiego.
- **Dane transakcyjne:** Data wypożyczenia, Data zwrotu (rzeczywista).

2.3.2 Prototyp CSV

Aby zweryfikować kompletność przetwarzanych informacji, przygotowano “płaską” (nieznormalizowaną) reprezentację danych dla operacji wypożyczenia, bazującą na pierwotnych wytycznych.

```
Imie,Nazwisko,Ulica,Kod_Pocztowy,Miasto,Data_Zapisu,Tytul_Ksiazki,Imie_Autora,
↳Nazwisko_Autora,Kategoria,Data_Wypozyczenia,Data_Zwrotu
Jan,Kowalski,Kwiatowa 1,00-001,Warszawa,2024-01-15,Wiedźmin,Andrzej,Sapkowski,
↳Fantastyka,2024-03-01,2024-03-15
Jan,Kowalski,Kwiatowa 1,00-001,Warszawa,2024-01-15,Pan Tadeusz,Adam,Mickiewicz,Poezja,
↳2024-03-10,
Anna,Nowak,Polna 2,31-444,Kraków,2023-11-05,Lśnienie,Stephen,King,Horror,2024-02-20,
↳2024-03-05
```

2.3.3 Model Konceptualny (Pojęciowy)

Na podstawie analizy procesów i zebranych danych opracowano model pojęciowy, identyfikując obiekty, ich cechy oraz powiązania.

Zidentyfikowane encje

- **Czytelnik** - osoba zapisana do biblioteki.
- **Autor** - twórca dzieła literackiego.
- **Książka** - oferowana pozycja z inwentarza biblioteki.
- **Kategoria** - sekcja grupująca gatunkowo księgozbiór.
- **Wypożyczenie** - zdarzenie potwierdzające fakt wydania książki czytelnikowi.

Zdefiniowane atrybuty/własności

- **Czytelnik:** Imię, Nazwisko, Ulica, Kod pocztowy, Miasto, Data zapisu.
- **Autor:** Imię, Nazwisko.
- **Książka:** Tytuł, Rok wydania.
- **Kategoria:** Nazwa kategorii.
- **Wypożyczenie:** Data wypożyczenia, Data zwrotu.

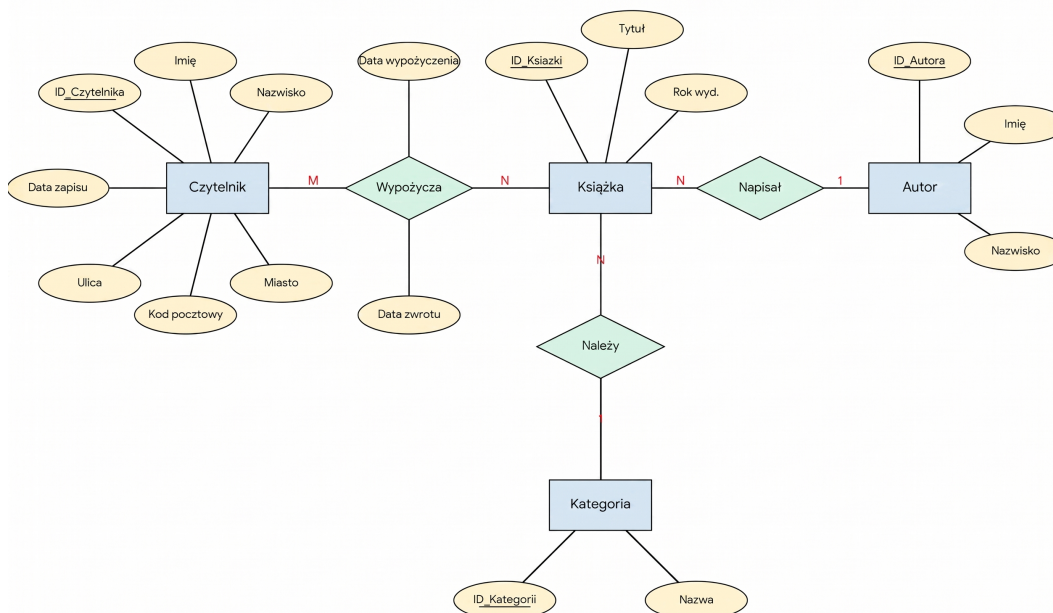
Opis związków

- **Autor – Książka (1:N):** Jeden autor może napisać wiele książek. (Dla uproszczenia modelu zakładamy głównego autora).
- **Kategoria – Książka (1:N):** Kategoria zawiera wiele tytułów.
- **Czytelnik – Wypożyczenie (1:N):** Czytelnik może dokonać wielu wypożyczeń w historii.
- **Książka – Wypożyczenie (1:N):** Jeden egzemplarz książki może być w swojej historii wypożyczany wielokrotnie różnym czytelnikom.

Identyfikacja encji słabych

Występuje relacja wiele-do-wielu (M:N) między Czytelnikiem a Książką (czytelnik czyta wiele książek, książka jest czytana przez wielu czytelników). Związek ten rozwiązywany jest przez encję asocjacyjną **Wypożyczenia**, która przechowuje wszystkie historyczne i bieżące transakcje między dwoma stronami. * **Uzasadnienie:** Byt ten nie istnieje samodzielnie w oderwaniu od konkretnego Czytelnika i konkretnej Książki.

Schemat w notacji Chena



2.3.4 Model logiczny i proces normalizacji

Przebieg procesu normalizacji

Krok 1: Pierwsza Postać Normalna (1NF) Wiersze są unikalne, tworzymy sztuczne klucze główne (ID_Wypozyczenia). Wartości w komórkach są atomowe. Pojawia się duża redundancja danych adresowych i książkowych.

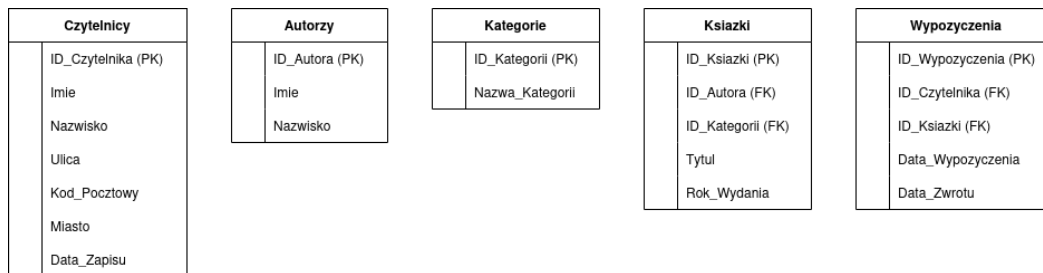
Krok 2: Druga Postać Normalna (2NF) Rozbijamy płaską strukturę względem częściowych zależności. Oddzielamy encje twarde od operacji. Tworzymy bazowe tabele: Czytelnicy oraz Książki. Powstaje tabela Wypozyczenia przechowująca klucze obce ID_Czytelnika oraz ID_Książki, a także daty transakcji. *Uwaga:* Dynamiczne atrybuty takie jak “suma wypożyczeń” z pierwotnego zadania są usuwane ze schematu, ponieważ łamią zasady normalizacji – można je obliczyć zapytaniem SQL typu COUNT(*) na zdarzeniach w tabeli Wypozyczenia.

Krok 3: Trzecia Postać Normalna (3NF) Eliminacja zależności przechodnich. Z tabeli Książki wydzielamy powtarzające się nazwy gatunków do tabeli Kategorie (klucz: ID_Kategorii) oraz dane twórców do tabeli Autorzy (klucz: ID_Autora). Każda tabela reprezentuje teraz niezależną encję o jasno określonym celu biznesowym.

Ostateczna struktura tabel (3NF)

- **Czytelnicy:** ID_Czytelnika (PK), Imie, Nazwisko, Ulica, Kod_Pocztowy, Miasto, Data_Zapisu
- **Autorzy:** ID_Autora (PK), Imie, Nazwisko
- **Kategorie:** ID_Kategorii (PK), Nazwa_Kategorii
- **Ksiazki:** ID_Ksiazki (PK), ID_Autora (FK), ID_Kategorii (FK), Tytul, Rok_Wydania
- **Wypozyczenia:** ID_Wypozyczenia (PK), ID_Czytelnika (FK), ID_Ksiazki (FK), Data_Wypozyczenia, Data_Zwrotu

Diagram ERD (Model Logiczny)



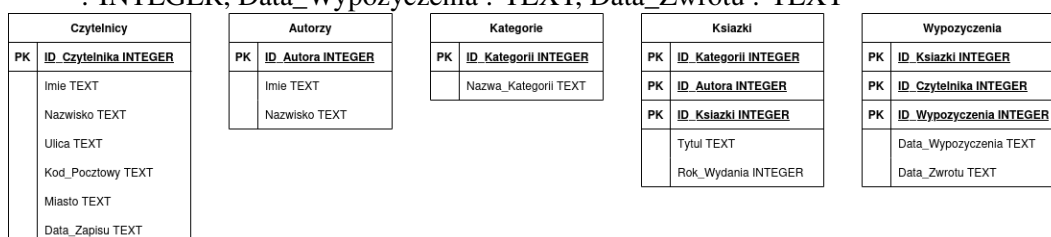
2.3.5 Model fizyczny bazy danych

Różnice implementacyjne między modelami fizycznymi wynikają wprost z silników RDBMS – ograniczeń typowania SQLite oraz bogatych możliwości deklaratywnych PostgreSQL.

Model fizyczny dla środowiska SQLite

Z uwagi na okrojony zestaw typów, daty są mapowane jako TEXT.

- **Czytelnicy:** ID_Czytelnika : INTEGER PRIMARY KEY, Imie : TEXT, Nazwisko : TEXT, Ulica : TEXT, Kod_Pocztowy : TEXT, Miasto : TEXT, Data_Zapisu : TEXT
- **Autorzy:** ID_Autora : INTEGER PRIMARY KEY, Imie : TEXT, Nazwisko : TEXT
- **Kategorie:** ID_Kategorii : INTEGER PRIMARY KEY, Nazwa_Kategorii : TEXT
- **Ksiazki:** ID_Ksiazki : INTEGER PRIMARY KEY, ID_Autora : INTEGER, ID_Kategorii : INTEGER, Tytul : TEXT, Rok_Wydania : INTEGER
- **Wypozyczenia:** ID_Wypozyczenia : INTEGER PRIMARY KEY, ID_Czytelnika : INTEGER, ID_Ksiazki : INTEGER, Data_Wypozyczenia : TEXT, Data_Zwrotu : TEXT



Model fizyczny dla środowiska PostgreSQL

PostgreSQL umożliwia zastosowanie precyzyjnych i natywnych typów, w tym rygorystycznych typów daty (DATE) oraz optymalizacji pamięciowej dla ciągów znaków (VARCHAR).

- **Czytelnicy:** ID_Czytelnika : SERIAL PRIMARY KEY, Imie : VARCHAR(50), Nazwisko : VARCHAR(50), Ulica : VARCHAR(100), Kod_Pocztowy : VARCHAR(6), Miasto : VARCHAR(50), Data_Zapisu : DATE DEFAULT CURRENT_DATE
- **Autorzy:** ID_Autora : SERIAL PRIMARY KEY, Imie : VARCHAR(50), Nazwisko : VARCHAR(50)
- **Kategorie:** ID_Kategorii : SERIAL PRIMARY KEY, Nazwa_Kategorii : VARCHAR(50)
- **Ksiazki:** ID_Ksiazki : SERIAL PRIMARY KEY, ID_Autora : INTEGER REFERENCES Autorzy, ID_Kategorii : INTEGER REFERENCES Kategorie, Tytul : VARCHAR(150), Rok_Wydania : SMALLINT

- **Wypożyczenia:** ID_Wypożyczenia : SERIAL PRIMARY KEY, ID_Czytelnika : INTEGER REFERENCES Czytelnicy, ID_Ksiazki : INTEGER REFERENCES Ksiazki, Data_Wypożyczenia : DATE DEFAULT CURRENT_DATE, Data_Zwrotu : DATE

Czytelnicy		Autorzy		Kategorie		Ksiazki		Wypożyczenia	
PK	ID_Czytelnika SERIAL	PK	ID_Autora SERIAL	PK	ID_Kategorii SERIAL	PK	ID_Kategorii INTEGER	PK	ID_Ksiazki INTEGER
	Imie VARCHAR(50)		Imie VARCHAR(50)		Nazwa_Kategorii VARCHAR(50)	PK	ID_Autora INTEGER	PK	ID_Czytelnika INTEGER
	Nazwisko VARCHAR(50)		Nazwisko VARCHAR(50)			PK	ID_Ksiazki SERIAL	PK	ID_Wypożyczenia SERIAL
	Ulica VARCHAR(100)						Tytul VARCHAR(150)		Data_Wypożyczenia DATE
	Kod_Pocztowy VARCHAR(6)						Rok_Wydania SMALLINT		Data_Zwrotu DATE
	Miasto VARCHAR(50)								
	Data_Zapisu DATE								

2.4 Rozdział 4: Implementacja fizyczna i zasilanie bazy danych

Autorzy

1. Paweł Łoćwin
2. Paweł Łosowski

2.4.1 Definicja fizyczna bazy danych (Zadanie 1)

Na podstawie opracowanych wcześniej modeli logicznych przygotowano skrypty DDL (Data Definition Language) dla dwóch docelowych silników relacyjnych baz danych: PostgreSQL oraz SQLite. Kody te przygotowały solidną fundamentę dla skutecznego przechowywania i zarządzania danymi biblioteki.

Skrypt dla środowiska PostgreSQL

Środowisko PostgreSQL pozwala na wykorzystanie zaawansowanych, natywnych typów danych oraz rygorystyczne egzekwowanie więzów integralności. W poniższym skrypcie kluczowym elementem jest użycie więzów klucza obcego (FOREIGN KEY) w celu zapewnienia spójności referencyjalnej między tabelami.

```
-- Fragment kodu definicji kluczowych tabel (PostgreSQL)
CREATE TABLE Ksiazki (
    ID_Ksiazki SERIAL PRIMARY KEY,
    ID_Autora INTEGER REFERENCES Autorzy(ID_Autora),
    ID_Kategorii INTEGER REFERENCES Kategorie(ID_Kategorii),
    Tytul VARCHAR(150) NOT NULL,
    Rok_Wydania SMALLINT
);

CREATE TABLE Wypożyczenia (
    ID_Wypożyczenia SERIAL PRIMARY KEY,
    ID_Czytelnika INTEGER REFERENCES Czytelnicy(ID_Czytelnika),
    ID_Ksiazki INTEGER REFERENCES Ksiazki(ID_Ksiazki),
    Data_Wypożyczenia DATE DEFAULT CURRENT_DATE,
    Data_Zwrotu DATE
);
```

Skrypt dla środowiska SQLite

W silniku SQLite struktura ulega pewnemu uproszczeniu z powodu mniejszej rygorystyczności typowania oraz bardzo ograniczonej liczby wbudowanych klas typów danych (np. brak odrębnego typu daty). Pomimo ograniczeń, SQLite pozostaje niezawodnym rozwiązaniem do prototypowania i mniejszych aplikacji.

```
-- Fragment kodu definicji kluczowych tabel (SQLite)
CREATE TABLE Wypożyczenia (
    ID_Wypożyczenia INTEGER PRIMARY KEY AUTOINCREMENT,
```

(continues on next page)

(continued from previous page)

```

ID_Czytelnika INTEGER,
ID_Ksiazki INTEGER,
Data_Wypozyczenia TEXT,
Data_Zwrotu TEXT,
FOREIGN KEY(ID_Czytelnika) REFERENCES Czytelnicy(ID_Czytelnika),
FOREIGN KEY(ID_Ksiazki) REFERENCES Ksiazki(ID_Ksiazki)
);

```

2.4.2 Mechanizm zasilania bazy danych (Zadanie 2)

Posiadając gotową strukturę bazy, należało wyposażać ją w dane demonstracyjne. Przygotowano zbiór danych testowych, które zostały poddane wsadowemu ładowaniu (batch import).

Formatyzacja danych (Pliki CSV)

Dane do importu przygotowano w niezależnym formacie płaskim CSV (Comma-Separated Values). Takie podejście pozwala na łatwą edycję danych poza systemem bazodanowym. Poniżej zaprezentowano wybrane kategorie książek przygotowane w formacie CSV.

```

Nazwa_Kategorii
Fantastyka
Kryminał
Literatura faktu
Poezja
Horror

```

Implementacja mechanizmu importu

Do zaimportowania powyższych danych wybrano mechanizm wprowadzania wielowartościowego oparty na technice **INSERT (multi-value/batch)**. Rozwiązanie zrealizowano z wykorzystaniem języka Python wraz z biblioteką `psycopg` do komunikacji z bazą PostgreSQL.

Wybór tego mechanizmu podyktowany jest kompromisem między uniwersalnością a wydajnością. Użycie metody `executemany()` na obiekcie kursora bazy danych pozwala na szybkie wstawienie wielu rekordów bez konieczności wykonywania zapytania dla każdego wiersza osobno.

Poniżej przedstawiono fragment skryptu aplikacyjnego odpowiedzialnego za odczyt z pliku i zasilenie tabel:

```

import csv
import psycopg

# 1. Wczytywanie danych z pliku CSV do struktury iterowalnej (listy krotek)
def wczytaj_dane_csv(sciezka_pliku):
    dane = []
    with open(sciezka_pliku, mode='r', encoding='utf-8') as plik:
        csv_reader = csv.reader(plik)
        next(csv_reader) # Pominięcie wiersza nagłówka
        for wiersz in csv_reader:
            dane.append(tuple(wiersz))
    return dane

# 2. Właściwy proces wsadowego importu do bazy PostgreSQL
def importuj_do_postgres(kategorie_do_importu, db_config):
    try:
        # Użycie Context Managera dla bezpiecznego zarządzania transakcjami

```

(continues on next page)

(continued from previous page)

```

with psycopg.connect(**db_config) as conn:
    with conn.cursor() as cursor:

        # Definicja sparametryzowanego zapytania zapobiegającego SQL Injection
        zapytanie_sql = "INSERT INTO Kategorie (Nazwa_Kategorii) VALUES (%s)"

        # Wsadowe wywołanie zapytań (batch insert)
        cursor.executemany(zapytanie_sql, kategorie_do_importu)

        print(f"Pomyślnie zaimportowano {len(kategorie_do_importu)} rekordów.
→ ")

except Exception as e:
    print(f"Wystąpił błąd operacji wsadowej: {e}")

```

2.4.3 Podsumowanie

Niniejszy rozdział wieńczy etap projektowania i wdrażania struktury relacyjnej bazy danych dla systemu zarządzania biblioteką. Poprzez transformację modelu logicznego do postaci fizycznej osiągnięto kompletne i funkcjonalne rozwiązanie.

Pierwsza część rozdziału koncentrowała się na **Definicji fizycznej bazy danych (Zadanie 1)**, gdzie dokonano implementacji schematów dla dwóch wiodących silników relacyjnych: PostgreSQL i SQLite, stanowiących praktyczną alternatywę w zależności od wymagań skalowalności oraz zasobów dostępnych w środowisku produkcyjnym. Druga część rozdziału poświęcona została **Mechanizmowi zasilania bazy danych (Zadanie 2)**. Dane demonstracyjne przygotowano w formacie CSV, który zapewnia elastyczność i łatwość edycji zarówno dla administratorów baz danych, jak i dla użytkowników końcowych systemu.

Automatyzacja procesu inicjalizacji systemu za pomocą skryptów Python stanowi znaczący wkład w praktyczność rozwiązania, umożliwiając łatwe replikowanie i testowanie bazy danych w różnych środowiskach bez konieczności wykonywania repetycyjnych czynności manualnych.

Niniejszy projekt stanowi kompletne studium cyklu życia relacyjnej bazy danych – od teoretycznego modelowania (Rozdziały 1-3) poprzez praktyczną implementację (Rozdział 4) – demonstrując całkowity proces od koncepcji do produkcyjnego systemu zarządzania danymi.

2.5 Rozdział 5: Komunikacja i operacje na danych

Autorzy

1. Paweł Łoćwin
2. Paweł Łosowski

2.5.1 Wstęp

Po zdefiniowaniu i zasileniu struktur bazodanowych w poprzednich rozdziałach, system wymaga interfejsu zdolnego do obsługi zaawansowanej logiki biznesowej biblioteki. W tym celu zaimplementowano warstwę dostępu do danych (Data Access Layer) w języku Python.

Wykorzystano techniki tworzenia rozbudowanych zapytań SQL, w tym: * Podzapytania (skorelowane i nieskorelowane w klauzulach SELECT, FROM, WHERE). * Złączenia relacyjne (JOIN, LEFT JOIN). * Analizę agregacyjną (GROUP BY, HAVING). * Operatory teorii mnogości (UNION, INTERSECT, EXCEPT).

2.5.2 Pełna specyfikacja stworzonego API

Poniżej przedstawiono zautomatyzowaną dokumentację modułu obsługującego połączenia z bazami PostgreSQL oraz SQLite, wygenerowaną na podstawie wbudowanych komentarzy (Docstrings).

Moduł biblioteka_zapytania

Moduł zawiera zestaw zaawansowanych zapytań SQL do obsługi systemu zarządzania biblioteką. Obsługuje dwa silniki bazodanowe: PostgreSQL oraz SQLite.

Wymagania: - psycopg2 (lub psycopg) dla PostgreSQL - sqlite3 (wbudowany) dla SQLite

`biblioteka_zapytania.get_aktywni_czytelnicy_powyzej_limitu(conn, limit_wypozycczen)`

Pobiera listę czytelników, którzy w historii wypożyczyli więcej książek niż wynosi podany limit. Wykorzystuje funkcje agregujące, złączenia oraz klauzulę HAVING.

Parameters

- `conn` (`psycopg2.extensions.connection`) – Obiekt aktywnego połączenia z bazą PostgreSQL (`psycopg2`).
- `limit_wypozycczen` (`int`) – Minimalna liczba wypożyczeń uprawniająca do znalezienia się na liście.

Returns

Lista krotek zawierająca imię, nazwisko i sumę wypożyczeń czytelnika.

Return type

`list[tuple]`

`biblioteka_zapytania.get_czytelnicy_bez_wypozycczen(conn)`

Pobiera dane czytelników, którzy zapisali się do biblioteki, ale nigdy nie wypożyczyli żadnej książki. Zastosowano tu operator zbiorowy EXCEPT w celu odjęcia zbioru aktywnych od wszystkich.

Parameters

`conn` (`psycopg2.extensions.connection`) – Obiekt aktywnego połączenia z bazą PostgreSQL.

Returns

Lista krotek z danymi czytelników (ID, Imię, Nazwisko).

Return type

`list[tuple]`

`biblioteka_zapytania.get_katalog_osob(conn)`

Zwraca ujednoliconą listę wszystkich osób figurujących w systemie, przypisując im odpowiednią rolę. Używa operatora zbiorowego UNION w celu złączenia tabel Czytelnicy i Autorzy.

Parameters

`conn` (`sqlite3.Connection`) – Obiekt połączenia z bazą SQLite.

Returns

Lista krotek postaci (Imię, Nazwisko, Rola).

Return type

`list[tuple]`

`biblioteka_zapytania.get_ksiazki_wypozycczone_w_miescie(conn, nazwa_miasta)`

Wyszukuje tytuły książek, które kiedykolwiek zostały wypożyczone przez mieszkańców określonego miasta. Rozwiązanie bazuje na podzapytaniu skorelowanym w klauzuli WHERE.

Parameters

- `conn` (`psycopg2.extensions.connection`) – Obiekt aktywnego połączenia z bazą PostgreSQL.
- `nazwa_miasta` (`str`) – Nazwa miasta do przefiltrowania czytelników.

Returns

Lista krotek zawierająca wyłącznie tytuły książek.

Return type

`list[tuple]`

`biblioteka_zapytania.get_ksiazki_z_iloscia_wypozycczen(conn, rok_od)`

Zwraca szczegółowe dane o książkach wydanych po zadanym roku wraz z ilością ich historycznych wypożyczeń. Wykorzystuje funkcje wierszowe (UPPER) oraz podzapytanie w klauzuli SELECT.

Parameters

- **conn** (*psycopg2.extensions.connection*) – Obiekt aktywnego połączenia z bazą PostgreSQL.
- **rok_od** (*int*) – Dolna granica roku wydania książki (wyłącznie).

Returns

Lista krotek (Tytuł, ZWielkiejLitery_NazwiskoAutora, Total_Wypozycczen).

Return type

list[tuple]

`biblioteka_zapytania.get_najdluzej_przetrzymywane(conn, limit_rekordow)`

Wyszukuje wypożyczenia, które aktualnie trwają (brak daty zwrotu) i sortuje je rosnąco od najstarszych. Wykorzystuje funkcję wierszową DATE() silnika SQLite działającą na ciągach TEXT.

Parameters

- **conn** (*sqlite3.Connection*) – Obiekt połączenia z bazą SQLite.
- **limit_rekordow** (*int*) – Liczba rekordów do zwrócenia (LIMIT).

Returns

Lista krotek (Tytuł, Imię, Nazwisko, Data wypożyczenia).

Return type

list[tuple]

`biblioteka_zapytania.get_puste_kategorie(conn)`

Pobiera nazwy kategorii, do których aktualnie nie przypisano żadnych książek w inwentarzu. Demonstruje wykorzystanie LEFT JOIN w celu znalezienia brakujących relacji (NULL).

Parameters

conn (*sqlite3.Connection*) – Obiekt połączenia z bazą SQLite.

Returns

Lista krotek z nazwami “pustych” kategorii.

Return type

list[tuple]

`biblioteka_zapytania.get_raport_autorow(conn)`

Tworzy zaawansowany raport o autorach, zliczając ich wszystkie książki oraz wskazując tytuł chronologicznie najnowszego dzieła za pomocą podzapytania w SELECT.

Parameters

conn (*sqlite3.Connection*) – Obiekt połączenia z bazą SQLite.

Returns

Lista krotek (Imię autora, Nazwisko, Suma dzieł, Tytuł najnowszej książki).

Return type

list[tuple]

`biblioteka_zapytania.get_srednia_wyposycczen_na_czytelnika(conn)`

Oblicza średnią liczbę wypożyczonych książek przypadającą na jednego aktywnego czytelnika. Wykorzystuje podzapytanie w klauzuli FROM do wcześniejszej agregacji danych.

Parameters

conn (*sqlite3.Connection*) – Obiekt połączenia z bazą SQLite.

Returns

Zwraca jedną krotkę z pojedynczą wartością zmiennoprzecinkową (średnia).

Return type

list[tuple]

`biblioteka_zapytania.get_wszechstronni_czytelnicy(conn)`

Zwraca czytelników (Imię i Nazwisko), którzy wypożyczali książki z kategorii ‘Fantastyka’ oraz jednocześnie z kategorii ‘Kryminał’. Używa operatora zbiorowego INTERSECT.

Parameters

conn (*psycopg2.extensions.connection*) – Obiekt aktywnego połączenia z bazą PostgreSQL.

Returns

Lista krotek z imionami i nazwiskami wszechstronnych czytelników.

Return type
list[tuple]